Application Security Methodology Final Report

Haleemah Anwary

Mark Shtern

August 14, 2026

Abstract

This study replicates the proof-of-concept (PoC) proposed in "Enhancing Zero-Trust Security in Kubernetes" by Nethmee Udara Amadoru in a local Kubernetes environment. The study's aim is to verify the original results, test the approach's reproducibility in the local environment, and evaluate its strengths and limitations. The PoC's custom controller scripts were adapted for deployment in a local Kubernetes cluster deployed through kind, achieving real-time threat detection and automated response as originally proposed. The replication was evaluated through functional validation and log analysis by introducing simulated runtime threats and observing the controller's behavior to verify that threats are detected and mitigated as expected. The findings confirm that the PoC's zero-trust enforcement mechanism can quickly mitigate threats with minimal human intervention but also reveal current limitations.

In addition to replicating the original thesis, we further extended the proposed Zero Trust runtime framework by introducing application-layer security enforcement. A Web Application Firewall using ModSecurity was added into the Kubernetes environment and controlled dynamically through intent-based automation. A directory traversal attack was used as the evaluation scenario to test the runtime detection and response. A custom controller monitored server logs, validated enforcement authorization through explicit intents, and automatically added defensive rules into the firewall configuration upon detection. The experimental results showed that attacks were unsuccessful following remediation while all authorized network traffic continued to function normally. This experiment shows that Zero Trust security principles work to enhance application runtime protection through application-level controls which extend past network security measures.

Table of Contents

Abstract....2

1. Introduction....7

2. Literature Review....9

3. Methodology....13

3.1 Overview....13

3.2 Research Design....13

3.3 Original Framework....14

3.3.1 Overview of the Original Framework....14

3.3.2 Framework Components....15

3.3.3 Limitations of the Original Framework....16

3.4 Proposed Framework....17

3.4.1 Overview of the Proposed Framework....17

3.5 Chapter Summary....20

4. Analytical and Evaluation Methods....21

4.1 Data Collection....21

5. Local Implementation Details....23

5.1 Environment....23

5.2 Deployment....23

5.2.1 Pre-requisite Packages Installation....23

5.2.2 Creating Cluster through Kind....25

5.2.3 Installing Otterize....26

5.2.4 Deploying Server, Client-Other and Client using all.yaml....27

5.2.5 Applying Role Based Access Controls in Kubernetes....28

5.2.6 Executing Single-Run Controller Script....29

5.2.7 Deploying Continuous Watch Controller....31
5.3 ModSecurity WAF Deployment Architecture....31
5.3.1 Custom ModSecurity Rule Management....32
5.3.2 Runtime Traversal Controller Design....32
5.3.3 Intent-Controlled Enforcement Logic....33
5.3.4 RBAC and Least Privilege Configuration....33
6. Evaluation & Results....34
6.1 Analyzing/Testing Environment....34
6.1.1 Kubernetes cluster config:....34
6.1.2 Otterize Integration....36
6.1.3 Initial Application Workload Configuration....37
6.2 Single Run Script Results....39
6.3 Resetting Environment:....42
6.4 Continuous Watch Script:....43
6.5 Application-Layer Attack Evaluation and Runtime Response....46
6.5.1 Baseline System Behavior Before Controller Activation....46
6.5.2 Directory Traversal Attack Before Runtime Enforcement....46
6.5.3 Controller Deployment and Intent Activation....47
6.5.4 Runtime Detection and Automated Remediation....48
6.5.5 Post-Remediation Behavior....48
6.5.6 Evaluation Summary....49
7. Critical Analysis....50
7.1 Strengths and Weaknesses of the Proposed Solution....50
7.1.1 Strengths....50
7.1.2 Weaknesses....50

7.2 Reproducibility, Scalability, and Practicality of the solution 51
7.2.1 Reproducibility: Adaptability to Local Environments 51
7.2.2 Practicality of the Approach 52
7.2.3 Scalability 53
7.3 Critical Analysis of the Application-Layer Runtime Extension 54
7.3.1 Effectiveness of Runtime Application-Layer Enforcement 54
7.3.2 Operational Tradeoffs and Limitations 55
7.3.4 Implications for Zero Trust Resilience 55
8. Conclusion and Future Work 56
8.1 Conclusion 56
8.2 Future Work 57
References 59
A. Appendix 61
A.1 Test Application 61
A.1.1 Application Workload Manifests 61
A.1.2 ClientIntent Manifest: 63
A.1.3 Dockerization 65
A.1.4 Role Based Access Control Rules 66
A.2 Automation Code 67
A.2.1 Imports and Environment variables: 67
A.2.2 Helper Functions 67
A.2.3 Single Run Automation 71
A.2.4 Continuous Deny Automation 74
A.2.5 Job/Deployment files: 81
A.3 Additional Scripts: 83

A.3.1 Resetting Environment to Baseline.....83

1. Introduction

Cloud-native applications are becoming more common, and Kubernetes has become one of the most popular platforms for deploying and managing containerized applications. As more organizations move their applications to Kubernetes, protecting these environments has become increasingly important. Kubernetes environments are highly dynamic, with workloads constantly starting, stopping, scaling, and communicating with each other, creating security challenges that continue during runtime (Darwesh et al., 2022). Zero Trust Architecture (ZTA) is relevant to this type of environment because it follows the principle of "never trust, always verify." Instead of automatically trusting users or services based on their location in the network, ZTA requires access to be continuously evaluated based on factors such as identity, the resource being accessed, and the current security context (Rose et al., 2020).

Although Kubernetes provides several built-in security features such as Role-Based Access Control (RBAC), NetworkPolicies, and Pod Security Standards, these controls mainly enforce predefined access, communication, and workload security policies. They help reduce the attack surface and enforce security policies, but they do not by themselves detect malicious behavior and automatically respond when a threat occurs during runtime. This can create a delay between identifying suspicious activity and acting against it. Bagheri et al. (2024) explains that manually investigating security alerts can delay mitigation and give an attacker more time to cause damage, while aggressive responses can also disrupt legitimate services. This problem is also reflected in industry findings. The 2024 Red Hat State of Kubernetes Security Report found that 89% of surveyed organizations experienced at least one container or Kubernetes security incident in the previous 12 months, while 45% reported incidents occurring during runtime. These findings show why Kubernetes security requires not only preventive controls, but also runtime monitoring and an appropriate response when security conditions change.

Researchers have explored several approaches to improve runtime security in Kubernetes, with each study focusing on different aspects of threat detection and response. Cao et al. (2022) use network-flow data to detect abnormal behaviour in running workloads, while Joshi and Segireddy (2026) use eBPF telemetry and machine learning to provide broader runtime visibility. These approaches improve continuous monitoring, but their implemented systems focus mainly

on detection rather than automated remediation. Other studies connect detection more directly to response. Aly et al. (2025) combine runtime threat detection with adaptive deception, while Shenoy et al. (2025) use Falco and Kyverno to automatically delete a Pod after suspicious shell activity is detected. Bagheri et al. (2024) take a different approach by using risk-based mitigation while also considering the need to maintain service availability. Together, these studies show progress toward continuous monitoring and automated response, which are important for applying Zero Trust principles during runtime. However, they also show that the response taken after a threat is detected can vary in its level of granularity and effect on legitimate services. Although runtime security has advanced considerably, an important challenge remains in integrating runtime detection with Zero Trust-oriented enforcement and automated response in a way that provides effective containment without unnecessarily disrupting legitimate application access.

Against this broader research landscape, Amadoru (2025) proposes a Zero Trust-oriented approach that brings policy enforcement and automated remediation together within a Kubernetes environment. The Proof of Concept uses Otterize ClientIntents to define and enforce service-to-service access, while a custom controller monitors changes to these resources and automatically modifies the affected Deployment when the configured security condition is detected. This creates a direct link between changes in security conditions and automated enforcement without requiring manual intervention. However, the implemented remediation removes the application's exposed port, which blocks access to the service through that port rather than distinguishing between malicious and legitimate requests.

The proposed approach demonstrates how changes in security conditions, policy enforcement, and automated response can be integrated to strengthen Zero Trust security within a Kubernetes environment. However, its remediation strategy removes the application's exposed port once the configured security condition is detected, making the service unavailable through that port. While this effectively prevents further access through the exposed port, it may not be the most suitable approach for environments where maintaining service availability is equally important. To explore a more targeted alternative, this study replicates the original Proof of Concept in a local Kind-based Kubernetes environment and extends it by introducing application-layer protection using ModSecurity. Rather than removing the application's exposed port, the modified

approach detects directory traversal attacks through runtime log monitoring and dynamically applies ModSecurity WAF rules to block malicious requests while allowing legitimate traffic to continue.

2. Literature Review

Runtime security in Kubernetes involves more than identifying malicious activity after a workload has been deployed. Detection is only one part of runtime security; once suspicious behaviour is identified, an important question is whether and how the system should respond. Existing research approaches these two parts of runtime security in different ways: some studies focus on monitoring workload behaviour to detect potential threats during runtime (e.g., Cao et al., 2022), while others examine how detected threats can lead to active or automated responses (e.g., Bagheri et al., 2024). Together, this research shows that runtime security involves two closely connected questions: how malicious activity can be detected while workloads are running, and what action should follow once a threat has been identified.

Kubernetes runtime detection has been approached in different ways, depending on what kind of evidence a system is designed to observe. Cao et al. (2022) used probabilistic state machine models trained on NetFlow data to detect deviations from normal communication patterns, achieving a balanced accuracy of 99.2% and an F1 score of 0.982 across three multi-stage attack scenarios. An important strength of their approach is its interpretability, since the state-machine model provides a clearer view of why activity is considered anomalous compared with less interpretable neural-network models. Aly et al. (2025) also used network-flow data, but applied machine learning to classify several types of attacks, including path traversal, reporting accuracy between 86% and 91%. Joshi and Segireddy (2026) took a broader approach by combining eBPF monitoring of system calls, network activity, and file access with Kubernetes audit logs to assess runtime risk. Compared with the network-focused approaches of Cao et al. (2022) and Aly et al. (2025), this provides visibility across several layers of workload activity. Although these studies use different data and detection methods, they show that runtime threats can be identified from several forms of evidence, ranging from network communication patterns to activity observed across multiple layers of a Kubernetes environment.

Better detection, however, does not automatically lead to an automated response. This distinction can be seen across the detection approaches discussed above. Cao et al. (2022) focus on identifying anomalous behavior through learned state-machine models, but their framework does not extend to automated remediation. Similarly, Joshi and Segireddy (2026) provide broader runtime visibility through multi-layer telemetry and adaptive machine learning but identify automated remediation as an area for future work rather than the capability of their current system. Aly et al. (2025) go further by using detected threat information to deploy deceptive resources that redirect and engage attackers. However, this serves a different purpose from directly containing the malicious activity or remediating the affected workload. Together, these approaches show that detecting a threat, actively responding to it, and remediating it are related but distinct parts of runtime security. Improving detection therefore does not resolve the separate question of what action should be taken once malicious activity is identified.

Research on automated runtime response has explored different ways of containing threats once suspicious activity is detected, with important differences in how affected workloads are handled. Shenoy et al. (2025) demonstrate a reactive containment approach in which Falco detects suspicious shell activity, Falcosidekick forwards the alert to a webhook, and Kyverno automatically deletes the identified Pod. This creates a direct detection-to-remediation pipeline that reduces the need for manual intervention. However, the evaluation mainly focuses on shell-based threats, and the authors identify more granular responses, such as Pod isolation, as an area for future work. ACE-WARP takes a different approach by focusing on proactive mitigation and service availability. It uses Falco alerts to predict how an attack may progress, assesses the risk to other Pods, and migrates workloads considered at risk before subsequent attack stages occur (Bagheri et al., 2024). Rather than deleting the workload, ACE-WARP uses non-disruptive mitigation through Pod migration to reduce risk while maintaining service availability, although migration can introduce some delays. Together, these approaches show that automated containment involves more than responding quickly to a detected threat; the operational impact of the response and its effect on service availability must also be considered.

Beyond the operational impact of a response, its level of granularity is also important. The approaches discussed above are designed around different threat models and therefore respond at different levels, rather than providing a single form of containment that is appropriate for every

type of threat. This becomes particularly important when malicious behavior occurs within a specific application request. In this case, a workload-level response may affect more of the application than is necessary to address the malicious activity itself. For application-layer attacks, a more targeted approach could instead act on the malicious request itself while allowing legitimate application activity to continue. This creates a reason to examine application-layer enforcement as another form of runtime response.

Application-layer security provides another way to make runtime responses more targeted because individual malicious requests can be identified and blocked without removing or relocating the workload itself. Díaz-Verdejo et al. (2024) developed InspectorLog, which analyzes HTTP server logs using signature rules from tools such as ModSecurity and Snort. Instead of examining container or cluster behavior, the approach focuses on the URI in individual HTTP requests to identify malicious request patterns. However, InspectorLog is designed for offline analysis, so it supports detection and investigation rather than real-time enforcement. ModSecurity takes this further by allowing rules to block HTTP requests in real time. MK et al. (2026) found that ModSecurity with the OWASP Core Rule Set performed well against direct attacks, including path traversal, but was less effective against encoded and obfuscated payloads, with 264 of 669 malicious requests initially bypassing the default rules. Adding custom rules greatly reduced these bypasses, showing that ModSecurity can provide targeted enforcement, but its effectiveness depends on how well the rules are configured. Together, these studies show that application-layer security can operate at the level of individual requests: application logs can provide evidence of malicious behaviour, while WAF rules can block matching requests without acting against the entire workload.

Zero Trust requires security decisions to be continuously evaluated while limiting access according to least-privilege principles (Rose et al., 2020). The studies discussed above address different parts of these Zero Trust principles. Runtime detection approaches provide evidence about changing security conditions, while automated response approaches take action once a threat is identified. Application-layer approaches can provide more targeted enforcement by acting on individual malicious requests. However, the studies reviewed here generally examine these capabilities separately rather than combining runtime threat detection with targeted automated enforcement. Together, this leaves an important challenge: connecting runtime threat

detection with a Zero Trust-oriented response that can contain malicious activity while preserving legitimate access.

Against this broader research landscape, Amadoru (2025) proposes a Zero Trust-oriented approach that brings policy enforcement and automated remediation together within a Kubernetes environment. The Proof of Concept uses Otterize ClientIntents to define and enforce service-to-service access, while a custom controller monitors changes to these resources and automatically modifies the affected deployment when the configured security condition is detected. This creates a direct link between changes in the monitored resources and automated enforcement without requiring manual intervention. However, the implemented remediation removes the application's exposed container port, which prevents further communication through that port. Amadoru also identifies the focus on eliminating a single port as a limitation of the study.

This raises a more specific question that follows from the research discussed above: whether threat containment can be made more granular while still preserving legitimate access to the application and supporting Zero Trust principles. Bagheri et al. (2024) show the importance of service continuity during automated mitigation, while Díaz-Verdejo et al. (2024) show that malicious HTTP behaviour can be identified by monitoring application logs and examining request information such as the URI. MK et al. (2026) further show that ModSecurity rules can be used to selectively block malicious HTTP requests, including path-traversal attacks. Together, these findings point to a variation of Amadoru's approach worth testing, monitoring application logs for evidence of a directory-traversal attack and using that evidence to trigger targeted ModSecurity enforcement rather than removing the application's exposed port. This allows the study to examine whether runtime evidence can be used to automatically contain malicious requests while preserving legitimate access and maintaining the Zero Trust principles of continuous evaluation and least-privilege enforcement.

3. Methodology

3.1 Overview

This chapter describes the methodology used to investigate the research problem identified in Chapter 2 by replicating and extending the Proof of Concept proposed by Amadoru (2025). It first explains the original Zero Trust-oriented framework before presenting the modifications introduced in this study to support more targeted runtime threat containment. The chapter then describes the proposed framework, its main components, and the methodological decisions that guided its design. Together, these sections explain how the study investigates whether application-level evidence can be used to trigger automated ModSecurity enforcement while preserving legitimate application access and supporting the Zero Trust principles of continuous evaluation and least-privilege enforcement.

3.2 Research Design

This study uses a replication-and-extension research design based on the Proof of Concept proposed by Amadoru (2025). A replication-and-extension approach was selected because it allows the original methodology to be reproduced before introducing and evaluating a targeted modification within the same overall framework. This makes it possible to examine the effect of the proposed change while preserving the core design and objectives of the original study.

The original framework was first replicated within a local Kind-based Kubernetes environment to reproduce its Zero Trust policy enforcement and automated remediation workflow. Building on this implementation, the study introduces a targeted modification that replaces the original remediation strategy with application-layer enforcement using ModSecurity. Rather than removing the application's exposed container port after a configured security condition is detected, the modified framework monitors application logs for evidence of a directory traversal attack and uses this evidence to automatically apply ModSecurity rules that selectively block malicious requests while allowing legitimate traffic to continue.

This research design allows the proposed modification to be evaluated within the same architectural framework as the original Proof of Concept while addressing the research question identified in Chapter 2. By preserving the overall framework and changing only the remediation

mechanism, the study investigates whether application-level evidence can support a more targeted automated response that maintains service availability while remaining consistent with the Zero Trust principles of continuous evaluation and least-privilege enforcement.

3.3 Original Framework

3.3.1 Overview of the Original Framework

Amadoru (2025) proposes a Zero Trust-oriented framework that combines policy enforcement with automated remediation to strengthen Kubernetes runtime security. The framework was designed to reduce the need for manual intervention by linking predefined security conditions directly to automated enforcement within the Kubernetes environment. Rather than relying on an administrator to respond after a security condition is identified, the framework automatically applies the configured response through Kubernetes resources.

To achieve this, the framework uses Otterize ClientIntents to define service-to-service communication policies while a custom Kubernetes controller continuously monitors these resources for changes that represent predefined security conditions. When a configured condition is detected, the controller automatically modifies the affected Deployment to enforce the required security response. Kubernetes then reconciles the updated Deployment, allowing the configured security action to be applied automatically. This creates an end-to-end workflow that connects policy enforcement with automated remediation while remaining consistent with Zero Trust principles. Figure 1 illustrates the overall workflow of the original framework.

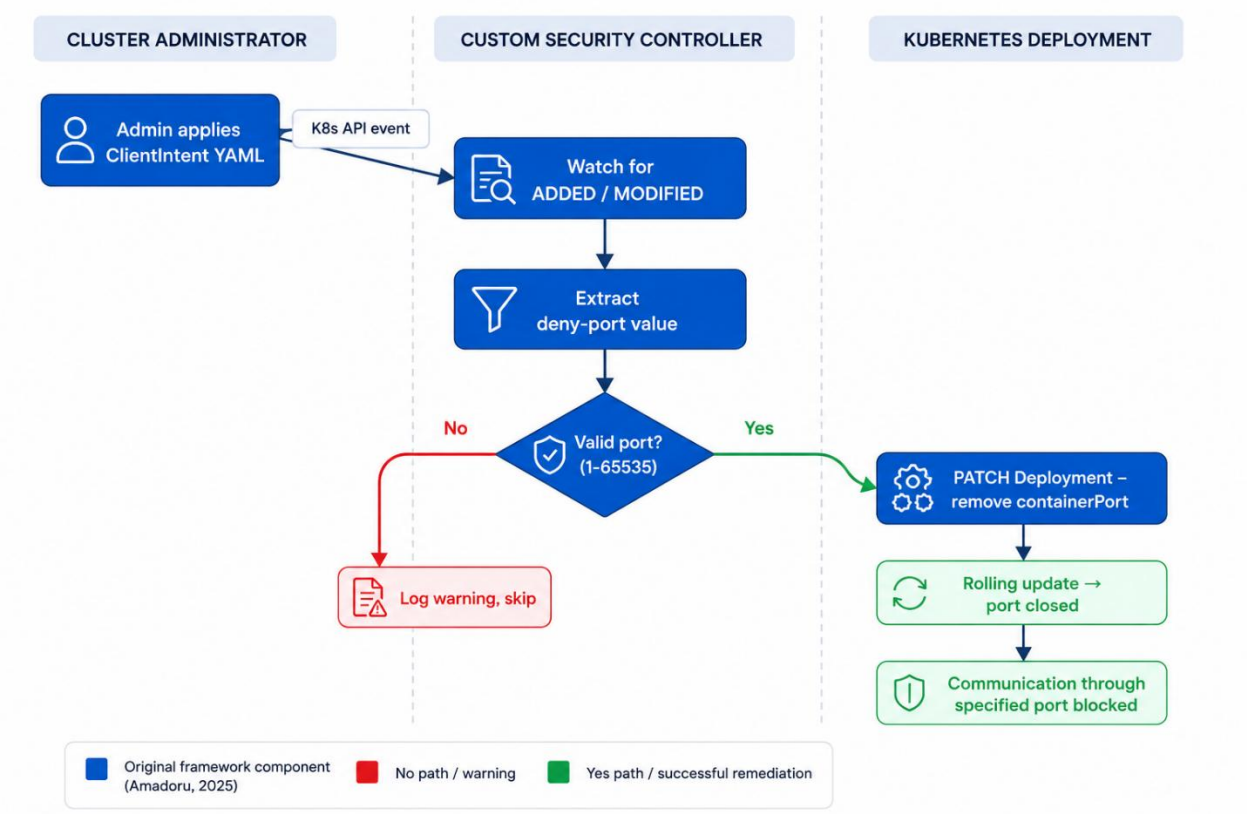


Figure 1: Original workflow of the Proof of Concept proposed by Amadoru (2025).

3.3.2 Framework Components

The original framework consists of three main components that work together to automate policy enforcement and remediation within the Kubernetes environment. As shown in Figure 1, the workflow moves from the Cluster Administrator to the Custom Security Controller, and finally to the Kubernetes Deployment. Each component performs a specific role in translating the configured security condition into an automated response.

Cluster Administrator

The Cluster Administrator initiates the workflow by applying the ClientIntent YAML to the Kubernetes cluster. The ClientIntent defines the intended communication policy and contains the information used by the framework to determine the required security response. Once the resource is applied, the Kubernetes API generates an event that can be observed by the Custom Security Controller.

Custom Security Controller

The Custom Security Controller monitors ClientIntent resources for ADDED or MODIFIED events. When one of these events occurs, the controller extracts the configured deny-port value and validates that it represents a valid port number between 1 and 65535. If the value is invalid, the controller logs a warning and skips the remediation. If the value is valid, the controller proceeds with the automated response by patching the Kubernetes Deployment to remove the specified containerPort.

Kubernetes Deployment

The Kubernetes Deployment applies the configuration change initiated by the Custom Security Controller. After the specified containerPort is removed, Kubernetes performs a rolling update so that the new Pod reflects the modified configuration. Once this process is complete, communication through the specified port is blocked. This completes the automated remediation workflow shown in Figure 1 without requiring the Cluster Administrator to manually modify the Deployment.

3.3.3 Limitations of the Original Framework

A limitation of Amadoru's (2025) Proof of Concept is the scope of its remediation strategy. When remediation is triggered, the framework removes the exposed containerPort, preventing further communication through that access path. Although this provides automated containment, it does not distinguish between malicious and legitimate traffic using the same application service. Amadoru (2025) also identifies the focus on eliminating a single port as a limitation and suggests that future work could explore more advanced automated responses.

This limitation is particularly relevant to application-layer attacks such as directory traversal, where malicious behaviour may be contained within individual HTTP requests while other application traffic remains legitimate. This creates an opportunity to investigate a more targeted remediation approach that addresses malicious behavior without removing access to the application itself.

3.4 Proposed Framework

3.4.1 Overview of the Proposed Framework

The proposed framework extends the methodology developed by Amadoru (2025) by introducing a more targeted approach to automated runtime response. The modification was designed to address the limitation identified in the original Proof of Concept, where removing the application's exposed container port prevents both malicious and legitimate communication through that port. Rather than changing the overall Zero Trust-oriented architecture, the proposed framework preserves the original automated policy enforcement process while introducing a different remediation strategy.

Instead of linking remediation solely to changes in ClientIntent resources, the modified framework uses application-level evidence obtained through runtime log monitoring to determine when remediation should occur. When evidence of a directory traversal attack is identified, the framework dynamically updates ModSecurity rules to selectively block malicious HTTP requests rather than removing the application's exposed container port. This allows the framework to automatically contain malicious activity while maintaining legitimate application access.

The overall architecture continues to rely on a custom Kubernetes controller to coordinate automated policy enforcement. By replacing workload-level remediation with application-layer enforcement, the proposed framework investigates whether runtime application-level evidence can support a more targeted automated response while remaining consistent with the Zero Trust principles of continuous evaluation and least-privilege enforcement. Figure 3.2 illustrates the workflow of the proposed framework.

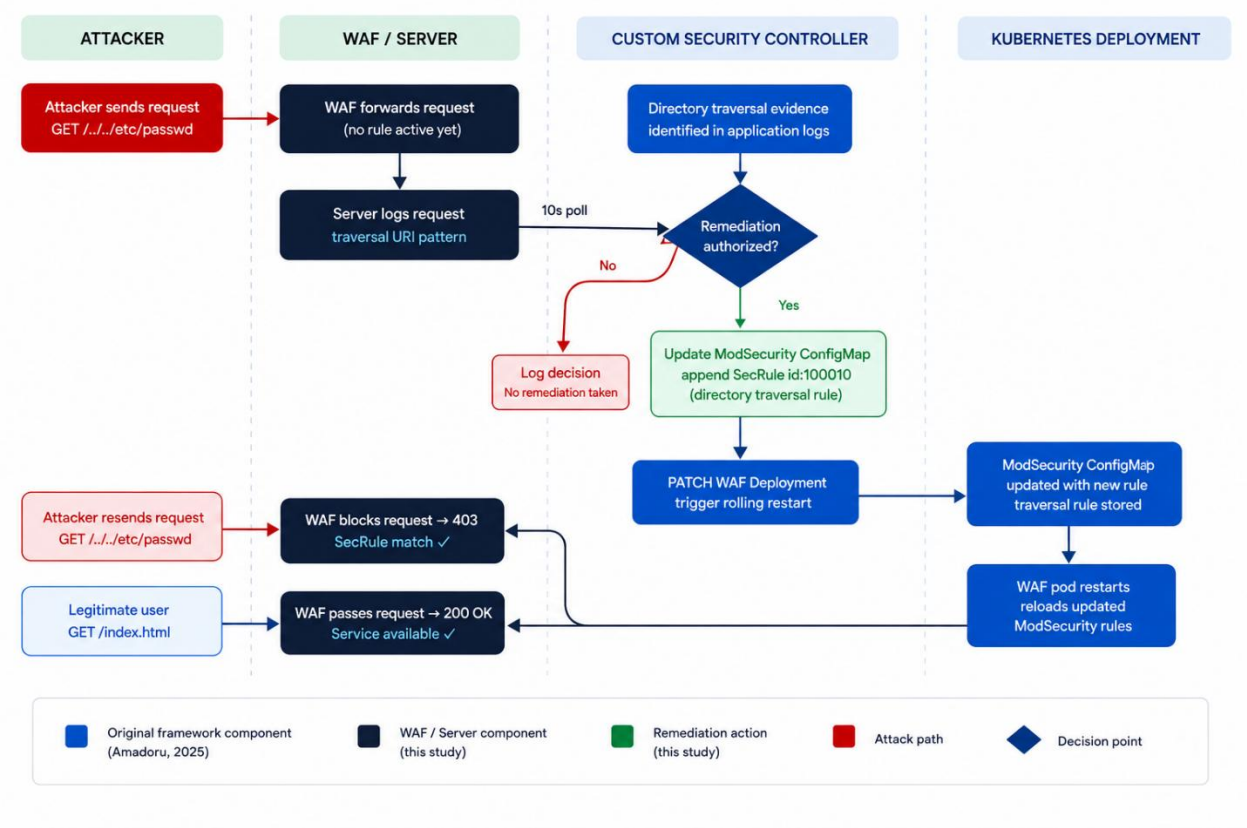


Figure 2: Modified workflow of the proposed framework for targeted runtime enforcement using application-level evidence.

3.4.2 Framework Components

The proposed framework extends the original automated remediation approach by introducing application-level detection and targeted enforcement. As shown in Figure 2, the workflow moves from the Attacker, through the WAF / Server, to the Custom Security Controller, and finally to the Kubernetes Deployment. The framework uses application logs as runtime evidence, ClientIntents to authorize remediation, and ModSecurity to enforce the resulting security rule.

Attacker

The Attacker initiates the workflow by sending a directory traversal request to the application, such as a request containing ../../etc/passwd. Before a targeted rule has been activated, the request is forwarded through the WAF to the application. This initial request provides the malicious activity that the framework is designed to identify and respond to.

WAF / Server

The WAF / Server handles incoming HTTP requests and provides the application-level evidence used by the framework. When the initial directory traversal request reaches the application, the server records the requested URI in its application logs. These logs allow the traversal pattern to be identified from the HTTP request itself rather than relying only on changes to Kubernetes resources.

After remediation has been completed, the WAF / Server also performs the targeted enforcement. A repeated directory traversal request that matches the newly applied ModSecurity rule is blocked with an HTTP 403 response, while a legitimate request, such as GET /index.html, continues through the WAF and receives an HTTP 200 OK response. This allows the framework to evaluate whether malicious traffic can be contained while the application remains accessible to legitimate users.

Custom Security Controller

The Custom Security Controller coordinates the automated remediation process. As shown in Figure 2, it periodically checks the application logs for evidence of directory traversal activity. When a matching URI pattern is identified, the controller evaluates whether remediation is authorized before taking action.

ClientIntents are used during this decision process to preserve the policy-driven design of the original framework. Rather than serving as the event that initiates remediation, the ClientIntent determines whether the configured response is authorized after application-level evidence has been detected. If remediation is not authorized, the controller logs the decision and no remediation is performed.

If remediation is authorized, the Custom Security Controller updates the ModSecurity ConfigMap by appending the configured directory traversal SecRule. The controller then patches the WAF Deployment to trigger a rolling restart, allowing the updated ModSecurity configuration to become active without requiring manual intervention.

Kubernetes Deployment

The Kubernetes Deployment applies the updated enforcement configuration initiated by the Custom Security Controller. The ModSecurity ConfigMap stores the newly added directory traversal rule, and the WAF Deployment is restarted so that the WAF Pod reloads the updated ModSecurity rules.

Once the updated configuration is active, subsequent requests are evaluated against the new rule. Directory traversal requests matching the configured pattern are blocked by ModSecurity, while requests that do not match the rule continue to the application. The Kubernetes Deployment therefore remains part of the automated remediation workflow, but the resulting enforcement is applied through the WAF rather than by removing the application's exposed port.

3.5 Chapter Summary

This chapter described the methodology used to investigate the research problem identified in Chapter 2. It explained the original Proof of Concept proposed by Amadoru (2025), presented the modified framework developed in this study, and described the methodological changes introduced to support more targeted runtime threat containment. The next chapter describes the implementation of the proposed framework within a local Kind-based Kubernetes environment, including the tools, technologies, and configurations used to evaluate the proposed approach.

4. Analytical and Evaluation Methods

The proposed framework was evaluated using a combination of log analysis, Kubernetes object inspection, and HTTP request testing. The evaluation focused on determining whether application-level evidence of a directory traversal attack could trigger automated remediation and whether the resulting ModSecurity rule could block malicious requests while preserving legitimate access to the application.

The following analytical methods were used:

- **Log Analysis:** Application and Custom Security Controller logs were reviewed to confirm that directory traversal activity was identified, remediation decisions were made correctly, and the automated response process was executed as expected.
- **Kubernetes Object Inspection:** Relevant Kubernetes resources were inspected to verify that the ModSecurity ConfigMap was updated with the new security rule and that the WAF Deployment completed the required rolling restart.
- **HTTP Behaviour Analysis:** curl requests were used to compare application behaviour before and after remediation. Directory traversal requests were used to determine whether the malicious request was successfully blocked, while legitimate requests were used to confirm that the application remained accessible.
- **Error Diagnosis:** Controller logs, Kubernetes resource states, and HTTP responses were examined together when necessary to identify errors and verify that each stage of the automated remediation workflow was completed successfully.

The main evaluation criteria were whether the framework could detect the configured directory traversal pattern, automatically apply the appropriate ModSecurity rule, block subsequent malicious requests, and continue allowing legitimate application traffic.

4.1 Data Collection

Data was collected from the application, the Custom Security Controller, Kubernetes resources, and HTTP request tests. These sources provided evidence for each stage of the workflow shown in Figure 2.

- **Application Logs:** HTTP request information was collected from the application logs to determine whether directory traversal patterns were present and available as runtime evidence for the Custom Security Controller.
- **Custom Security Controller Logs:** Controller logs were collected to observe log polling, directory traversal identification, remediation authorization, ConfigMap updates, and the initiation of the WAF Deployment restart.
- **Kubernetes Resource States:** The ModSecurity ConfigMap and WAF Deployment were inspected to confirm that the directory traversal rule was added successfully and that the updated configuration was loaded by the WAF.
- **HTTP Request Results:** curl outputs were recorded before and after remediation. The initial directory traversal request was used to establish baseline application behaviour before the targeted ModSecurity rule was active. After remediation, an HTTP 403 response was used as evidence that the malicious request was blocked, while an HTTP 200 OK response to a legitimate request was used to confirm that normal application access remained available.

5. Local Implementation Details

This section describes the local implementation of the proposed framework presented in Chapter 3. The implementation extends the original Proof of Concept by incorporating application-log-based detection and targeted ModSecurity enforcement for directory traversal attacks. The environment was configured to support the complete automated workflow, including runtime detection, ClientIntent-based authorization, dynamic ModSecurity rule updates, and enforcement through the WAF.

5.1 Environment

The original study implemented its Proof of Concept using Google Cloud Platform (GCP) and Google Kubernetes Engine (GKE). In this study, the proposed framework was implemented entirely within a local virtualized environment running Ubuntu 22.04 Desktop. This provided a controlled environment in which the framework could be implemented and evaluated without relying on external cloud infrastructure.

A local Kind-based Kubernetes cluster was used to deploy the application, WAF, Custom Security Controller, and supporting Kubernetes resources required by the proposed framework. The local environment also provided direct access to application logs, Kubernetes resources, and controller output, allowing each stage of the automated remediation workflow to be observed during testing

5.2 Deployment

5.2.1 Pre-requisite Packages Installation

To replicate the setup locally, several essential packages were installed in the following order:

- **Docker Engine(v28.5.1):** Required for running containerized workloads and providing the underlying environment for kubectl.
- **Kubectl(v1.34.1):** Kubernetes core command-line tool used to manage clusters and interact with Kubernetes resources.

- **Kind(v0.30.0):** Used to create and manage a local Kubernetes cluster using Docker containers as nodes.
- **Helm(v3.19.0):** The original study employed Helm charts to package and deploy the custom controller. In this replication, Helm was retained only for installing Otterize, while the custom controller was deployed manually without converting it into a Helm chart.

```
haleemah@haleemah-virtual-machine:~$ docker version
Client: Docker Engine - Community
 Version:           28.5.1
 API version:       1.51
 Go version:        go1.24.8
 Git commit:        e180ab8
 Built:             Wed Oct  8 12:17:03 2025
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          28.5.1
  API version:      1.51 (minimum version 1.24)
  Go version:       go1.24.8
  Git commit:       f8215cc
  Built:            Wed Oct  8 12:17:03 2025
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v1.7.28
  GitCommit:        b98a3aace656320842a23f4a392a33f46af97866
 runc:
  Version:          1.3.0
  GitCommit:        v1.3.0-0-g4ca628d1
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
haleemah@haleemah-virtual-machine:~$
```

Figure 3: Docker version

```
haleemah@haleemah-virtual-machine:~$ kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.0
haleemah@haleemah-virtual-machine:~$
```

Figure 4: Kubectl Version

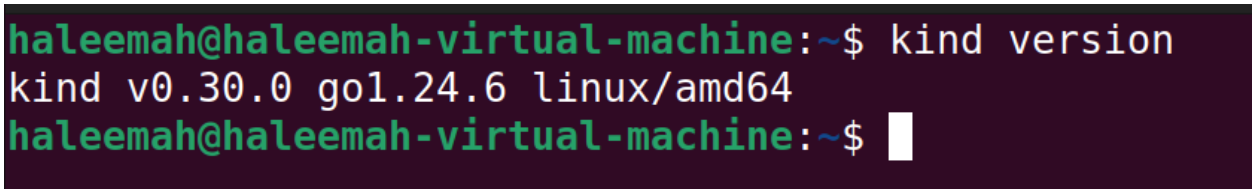


Figure 5: Kind Version

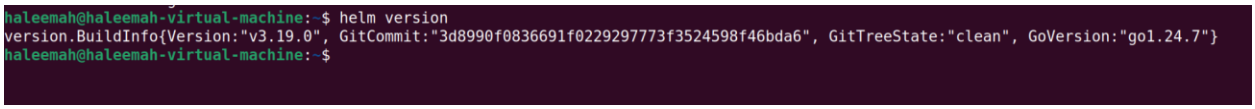


Figure 6: Helm Version

5.2.2 Creating Cluster through Kind

Create the cluster through kind. Run the command

```
kind create cluster --name otterize-local --image
kindest/node:v1.34.0
```

Cluster-Name: `otterize-local`

Image: `kindest/node:v1.34.0`

Version **v1.34.0** is used because the deployed **kubectl version** is **v1.34.1**, and the cluster version must be within one minor patch i.e., between v1.34.0 and v1.34.2 (*Install and Set up Kubectl on Linux*, 2025).

After creating cluster, verify by running

```
kubectl cluster-info
```

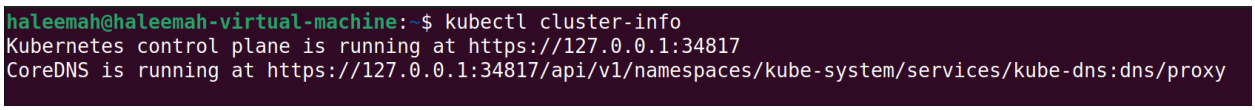


Figure 7: Kubernetes cluster info

5.2.3 Installing Otterize

Use Helm to install Otterize

```
helm repo add otterize https://helm.otterize.com

helm repo update

helm install otterize otterize/otterize-kubernetes -n
otterize-system --create-namespace --wait
```

Otterize Components Namespace: otterize-system

Verify that Otterize pods are ready before proceeding by using the `kubectl` command.

```
kubectl get pods -n otterize-system -o wide
```

If a pod keeps restarting, use the following command to view detailed information about it.

```
kubectl -n <namespace> describe pod <pod-name>
```

After installing the required packages and deploying Otterize, it is important to verify that the following core components are running successfully within the cluster. Each component serves a specific role in enforcing and visualizing Zero-Trust policies:

- **credentials-operator-controller:** Manages the lifecycle of service credentials, ensuring that each workload securely authenticates to the services it interacts with.
- **intents-operator-controller-manager:** Interprets and enforces `ClientIntent` resources by generating and applying the corresponding Kubernetes network policies.
- **otterize-network-mapper:** Continuously observes traffic patterns between services and provides a visual network map of service-to-service communication.
- **intents-operator-webhook-server:** Validates and processes incoming `ClientIntent` definitions through admission webhooks before they are applied to the cluster.
- **otterize-network-sniffer:** Captures live network traffic between workloads to detect implicit communication paths and update the network map accordingly.

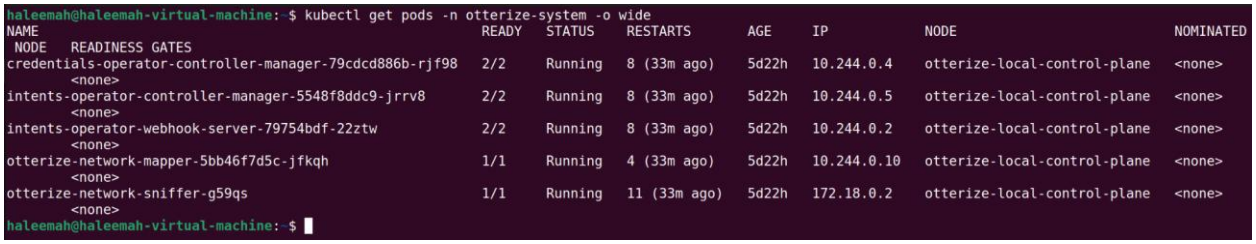


Figure 8: Pods in Otterize-System namespace

Verify that Otterize has created the ClientIntent custom resource named `clientintents.k8s.otterize.com`.

```
kubectl get crd clientintents.k8s.otterize.com
```

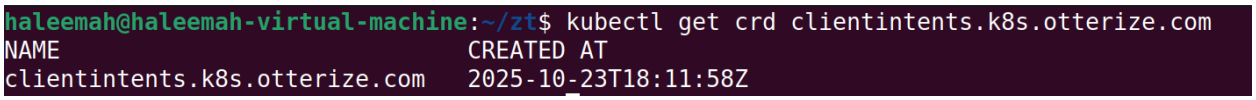


Figure 9: ClientIntnet CRD

5.2.4 Deploying Server, Client-Other and Client using all.yaml

The server, client, and client-other deployments were created using the `all.yaml` file provided in the Appendix of the original study. A minor modification was made to the server deployment by adding an additional port, 8080, to ensure that the custom controller does not unintentionally remove all exposed ports from a deployment during automated patching.

Configuration in all.yaml involves:

- Creates the `otterize-auto` namespace.
- Creates the `client` Deployment within the `otterize-auto` namespace.
- Creates the`client-other` deployment within the same namespace..
- Creates the `server` deployment in the `otterize-auto` namespace.
- Creates the `server1` service to expose the server deployment.

Command to apply `all.yaml`

```
kubectl apply -f all.yaml
```

Command to verify the running resources in the `otterize-auto` namespace

```
kubectl get deploy,svc,pods -n otterize-auto -o wide
```

```
haleemah@haleemah-virtual-machine:~$ kubectl get pods -n otterize-auto -o wide
NAME                            READY   STATUS             RESTARTS         AGE     IP           NODE                          NOMINATED NODE   READINESS
 GATES
client-79d48c7fbd-rglz4         0/1     ImagePullBackOff   3 (49m ago)      5d21h   10.244.0.9   otterize-local-control-plane   <none>           <none>
client-other-55498b9886-7v69c   0/1     ErrImagePull       3 (49m ago)      5d21h   10.244.0.6   otterize-local-control-plane   <none>           <none>
server-6dcdff7dbd-hp46p         0/1     CrashLoopBackOff   11 (4m42s ago)   22h     10.244.0.7   otterize-local-control-plane   <none>           <none>
haleemah@haleemah-virtual-machine:~$
```

Figure 10: Pods running in `otterize-auto` namespace

5.2.5 Applying Role Based Access Controls in Kubernetes

To allow the custom controller to interact securely with required Kubernetes resources, Role-Based Access Controls (RBAC) are applied using the `rbac.yaml` file. These controls define the permissions and access boundaries necessary for the controller to function correctly within the cluster.

The `rbac.yaml` configuration includes the following components:

ServiceAccount: defines `deny-automation-controller` which the controller pod uses to authenticate and communicate with the Kubernetes API.

Role: grants read access to `ClientIntents`, modify access to `Deployments`, and limited read access to `Pods` within the `otterize-auto` namespace .

RoleBinding: Links the ServiceAccount in the`otterize-automation` namespace with the Role in the `otterize-auto` namespace, enabling controlled cross-namespace operations.

Before applying `rbac.yaml` file, the `otterize-automation` namespace must be created, as it is referenced by the ServiceAccount definition:

```
kubectl create namespace otterize-automation
```

Once the namespace is created, apply the RBAC configuration:

```
kubectl apply -f rbac.yaml
```

5.2.6 Executing Single-Run Controller Script

The deployment of the single-run controller script involves containerizing the controller, loading it into the local Kubernetes cluster, and executing it as a one-time job. The following steps outline this process:

- **Containerizing the Controller:** The controller is packaged into a Docker image to ensure consistency and portability across environments..

```
docker build -t deny-auto:local .
```

- **Loading the Image into the Cluster:** The locally built image is loaded into the Kind-based Kubernetes cluster so it can be used by jobs or deployments.

```
kind load docker-image deny-auto:local --name
otterize-local
```

- **Applying the ClientIntent Resource:** The `deny-server-http-port` ClientIntent is applied before executing the script. Since the single-run version only reads existing `ClientIntent` objects once and then exits, this step must be completed beforehand.

```
kubectl apply -f clientintent.yaml
```

- **Running the Controller Job:** The controller image is executed as a one-time Kubernetes job within the `otterize-automation` namespace.

```
kubectl apply -f job-once.yaml
```

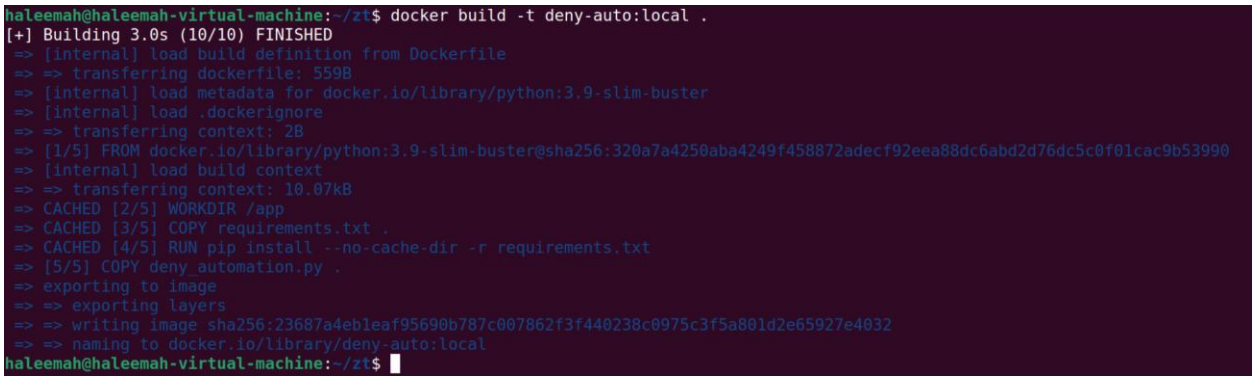


Figure 11: Docker image created for Custom controller Single Run Script

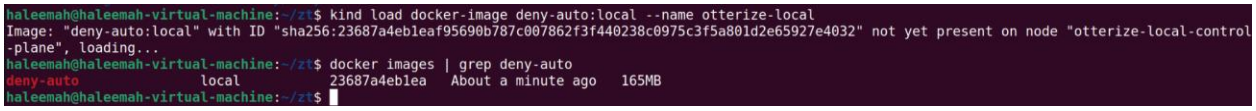


Figure 12: Custom Controller Single Run Script Image loaded in Kubernetes cluster

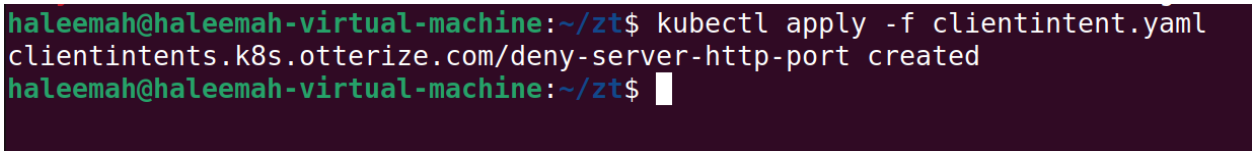


Figure 13: Client Intent Applied



Figure 14: Custom Controller Script running once through job

5.2.7 Deploying Continuous Watch Controller

The steps for building the Docker image, loading it into the cluster, and applying the ClientIntent remain identical to those used for the single-run controller. The main difference lies in how the controller is executed.

In the single-run setup, the controller was executed as a one-time Job that terminated after processing existing `ClientIntent` objects. In contrast, the Continuous Watch Controller is deployed as a Deployment, allowing it to run persistently within the cluster. This configuration enables continuous monitoring of the `ClientIntent` Custom Resource Definition (CRD) for any ADDED, MODIFIED, or DELETED events.

Because the Continuous Watch Controller operates in real time, the `ClientIntent` can be applied either before or after deploying the controller. The controller automatically detects and processes the resource whenever it appears.

Command to apply the deployment:

```
kubectl apply -f controller_deployment.yaml
```

```
haleemah@haleemah-virtual-machine:~/zt-2$ kubectl apply -f controller_deployment.yaml
deployment.apps/deny-automation-controller created
haleemah@haleemah-virtual-machine:~/zt-2$ kubectl logs -f deployment/deny-automation-controller -n otterize-automation
Starting Otterize Denial Automation (continuous watch) for namespace: otterize-auto
DEBUG: Attempting to establish watch on ClientIntents in namespace otterize-auto (Attempt 1)...
```

Figure 15: Deploying Continuous Watch Custom Controller

5.3 ModSecurity WAF Deployment Architecture

To extend the replicated Zero Trust framework to the application layer, a Web Application Firewall was introduced as an intermediary between external clients and the backend service. ModSecurity, combined with the OWASP Core Rule Set, was deployed using an Nginx-based container and configured as a reverse proxy. All external HTTP traffic was routed through this WAF, while the backend server was exposed only through an internal ClusterIP service.

This design ensured that application-layer inspection occurred before any request reached the server. The backend service remained unchanged, preserving the original application behavior while shifting inspection and enforcement responsibilities to the WAF layer. By isolating the

backend behind an internal service, direct access from external clients was prevented, enforcing a clear separation between public exposure and internal communication.

The WAF was configured to forward validated requests to the backend service using environment-based backend resolution. This allowed the deployment to remain flexible and consistent with Kubernetes service discovery while maintaining strict control over ingress traffic.

5.3.1 Custom ModSecurity Rule Management

Rather than relying solely on static rule sets, the proposed framework implemented dynamic ModSecurity rule management using a Kubernetes ConfigMap. A dedicated ModSecurity ConfigMap was created to store custom security rules and initially contained no active directory traversal protection. This allowed the framework to begin without the targeted traversal rule and enabled the security configuration to be updated at runtime without rebuilding container images or modifying the application code.

When a directory traversal attack was detected and remediation was authorized, the Custom Security Controller appended a ModSecurity rule targeting the identified traversal pattern to the ModSecurity ConfigMap. The rule was configured to deny matching requests and return an HTTP 403 response. Using the ConfigMap allowed the enforcement rule to be updated dynamically as part of the automated remediation workflow.

To activate the updated rule, the Custom Security Controller patched the WAF Deployment by modifying its Pod template annotation. This triggered a rolling restart of the WAF Pod, causing the updated ModSecurity configuration to be loaded. Once the restart was completed, subsequent HTTP requests were evaluated against the newly added rule.

5.3.2 Runtime Traversal Controller Design

The Custom Security Controller was implemented in Python to automate the detection and remediation workflow for directory traversal attacks. It was deployed as a Kubernetes Deployment and executed continuously within the cluster. Its primary responsibility was to monitor application logs generated by the server Pod for request patterns associated with directory traversal attempts.

The Custom Security Controller interacted with the Kubernetes API to identify the server Pod, retrieve recent application log entries, and evaluate them against predefined directory traversal patterns. When a matching pattern was identified, the controller recorded the detection event and initiated the authorization process before any remediation action was performed.

This log-based design allowed directory traversal activity to be identified from HTTP request information recorded during application runtime without requiring changes to the application code. Detection alone did not result in enforcement; instead, the identified activity was passed to the authorization stage of the workflow, where the corresponding ClientIntent was evaluated before remediation could proceed.

5.3.3 Intent-Controlled Enforcement Logic

To maintain policy-controlled automated remediation, enforcement was not performed immediately after a directory traversal pattern was detected. Instead, the Custom Security Controller evaluated an Otterize ClientIntent annotation to determine whether the configured remediation was authorized.

When a directory traversal attempt was identified, the Custom Security Controller queried the corresponding ClientIntent resource and evaluated its annotation value. If the annotation explicitly authorized traversal enforcement, the controller proceeded with remediation by adding the ModSecurity rule to the ModSecurity ConfigMap and triggering a rolling restart of the WAF Deployment. If remediation was not authorized, the controller logged the detection and authorization decision without modifying the security configuration.

This authorization step separated detection from enforcement within the automated workflow. A detected directory traversal attempts therefore resulted in remediation only when the corresponding ClientIntent permitted the response, allowing the framework to maintain policy-based control over automated security actions.

5.3.4 RBAC and Least Privilege Configuration

To support automated remediation while maintaining security boundaries, Kubernetes role-based access control (RBAC) was configured for the Custom Security Controller. A dedicated service

account was assigned to the controller, with permissions limited to the Kubernetes resources required for the detection, authorization, and remediation workflow.

The Custom Security Controller was granted read access to ClientIntent resources and Pod logs, access to read and modify the ModSecurity ConfigMap, and limited patch access to the WAF Deployment for triggering rolling restarts. These permissions allowed the controller to collect application-level evidence, evaluate remediation authorization, update the ModSecurity configuration, and activate new rules without granting broader access to unrelated Kubernetes resources.

This RBAC configuration applied least privilege access to the automated response process by limiting the Custom Security Controller to the permissions required for its assigned functions. This reduced the level of access granted to the automation component while still allowing the complete remediation workflow to operate.

6. Evaluation & Results

This section focuses on validating the functionality of the deployed components and evaluating the system's performance before and after applying the automated port denial patches. The process begins by verifying that all components, such as Otterize and the application deployments, are functioning as expected. Once the baseline environment is confirmed to be stable, patches are applied using both the single-run and continuous watch controller scripts.

6.1 Analyzing/Testing Environment

6.1.1 Kubernetes cluster config:

Before beginning the evaluation, it is essential to confirm that the Kubernetes cluster and its Role-Based Access Control (RBAC) configurations are functioning correctly. The first step is to verify that all required namespaces are present and properly configured. As illustrated in Figure 14, the environment includes the following namespaces and components:

- **otterize-auto**
    - Deployments: client, client-other, server.
    - Service: server

    - ClientIntents CRDs, including the deny-port example.

- **otterize-automation**
    - Custom controller deployment (deny-automation controller).
- **Otterize-system**
    - Otterize components such as the **Intents Operator**, **Network Mapper**, and other supporting services.

After confirming the namespace configuration, validate that the RBAC permissions are applied correctly by testing the following rules. These checks ensure that the `deny-automation-controller` ServiceAccount has the necessary privileges to interact with Kubernetes resources:

- Verify that the ServiceAccount can list ClientIntents in the `otterize-auto` namespace

```
kubectl auth can-i list clientintents --as
system:serviceaccount:otterize-automation:deny-
automation-controller -n otterize-auto
```

- Verify that the ServiceAccount can list Deployments in the`otterize-auto` namespace

```
kubectl auth can-i list deployments --as
system:serviceaccount:otterize-automation:deny-
automation-controller -n otterize-auto
```

- Verify that the ServiceAccount can patch Deployments in the`otterize-auto` namespace

```
kubectl auth can-i patch deployments --as
system:serviceaccount:otterize-automation:deny-
automation-controller -n otterize-auto
```

```
haleemah@haleemah-virtual-machine:~$ kubectl get namespaces
NAME                 STATUS   AGE
default              Active   10d
kube-node-lease      Active   10d
kube-public          Active   10d
kube-system          Active   10d
local-path-storage   Active   10d
otterize-auto        Active   10d
otterize-automation  Active   7d4h
otterize-system      Active   10d
```

Figure 16: Displaying Namespaces in the cluster

```
haleemah@haleemah-virtual-machine:~$ kubectl auth can-i list clientintents --as system:se
rviceaccount:otterize-automation:deny-automation-controller -n otterize-auto
yes
```

Figure 17: Verifying deny-automation-controller service account can list client intents

```
haleemah@haleemah-virtual-machine:~$ kubectl auth can-i list deployments --as system:serv
iceaccount:otterize-automation:deny-automation-controller -n otterize-auto
yes
haleemah@haleemah-virtual-machine:~$ kubectl auth can-i patch deployments --as system:ser
viceaccount:otterize-automation:deny-automation-controller -n otterize-auto
yes
haleemah@haleemah-virtual-machine:~$
```

Figure 18: Verifying deny-automation-controller service account can list and patch deployments

6.1.2 Otterize Integration

To ensure proper Otterize integration, two verifications are required: confirming that the core Otterize components are active, particularly the Intents Operator, and verifying that the `ClientIntent` Custom Resource Definition (CRD) has been successfully created.

Verify Otterize Components. List all running Otterize pods in the `otterize-system` namespace to confirm that the required components (such as the Intents Operator, Network Mapper, and Webhook Server) are operational.(shown in Figure 17)

```
Kubectl get pods -n otterize-system -o wide
```

Verify ClientIntent CRD Creation. Ensure that Otterize has successfully registered the `clientintents.k8s.otterize.com` CRD within the cluster. This confirms that the custom resources required for intent-based network policies are available.(shown in Figure 18)

```
kubectl get crd clientintents.k8s.otterize.com
```

```
haleemah@haleemah-virtual-machine:~$ kubectl get pods -n otterize-system -o wide
NAME                                                READY   STATUS    RESTARTS        AGE   IP           NODE                          NOMINATED
ODE   READINESS GATES
credentials-operator-controller-manager-79cdcd886b-rjf98   2/2     Running   16 (9m4s ago)   10d   10.244.0.2   otterize-local-control-plane   <none>
      <none>
intents-operator-controller-manager-5548f8ddc9-jrrv8       2/2     Running   16 (9m4s ago)   10d   10.244.0.5   otterize-local-control-plane   <none>
      <none>
intents-operator-webhook-server-79754bdf-22ztw             2/2     Running   16 (9m4s ago)   10d   10.244.0.3   otterize-local-control-plane   <none>
      <none>
otterize-network-mapper-5bb46f7d5c-jfkqh                   1/1     Running   8 (9m4s ago)    10d   10.244.0.6   otterize-local-control-plane   <none>
      <none>
otterize-network-sniffer-g59qs                             1/1     Running   15 (9m4s ago)   10d   172.18.0.2   otterize-local-control-plane   <none>
      <none>
haleemah@haleemah-virtual-machine:~$
```

Figure 19: Otterize Components

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl get crd clientintents.k8s.otterize.com
NAME                              CREATED AT
clientintents.k8s.otterize.com    2025-10-23T18:11:58Z
```

Figure 20: ClientIntnet CRD

6.1.3 Initial Application Workload Configuration

Before applying any automation scripts, the initial application workloads must be verified to ensure proper configuration and baseline connectivity. The following checks confirm that the server is correctly exposing both ports and that the client can successfully communicate with it over port 80.

Verify Server Deployment Configuration. Confirm that the `server` deployment exposes two ports,80 and 8080, by inspecting its YAML configuration.(shown in Figure 19)

```
kubectl get deployment server -n otterize-auto -o yaml
```

```
# To fetch ports from the output using grep
kubectl get deployment server -n otterize-auto -o yaml |
grep -A6 "ports:"
```

Test Client-to-Server Connectivity. Validate that the `client` pod can reach the `server` service on **port 80** using `curl`.(shown in Figure 20)

```
CLIENT_POD=$(kubectl get pod -n otterize-auto -l
app=client -o jsonpath='{.items[0].metadata.name}')

kubectl exec -n otterize-auto "$CLIENT_POD" -- sh -c 'curl
-v --max-time 3 server:80 || true'
```

Verify Server Logs. Check the logs of the `client` deployment to confirm that the requests are being sent to the `server`, indicating successful communication between the two pods.(shown in Figure 21)

```
kubectl logs -n otterize-auto deploy/client | tail -n 20
```

```
name: server
ports:
- containerPort: 80
  name: http
  protocol: TCP
- containerPort: 8080
  name: http-alt
  protocol: TCP
```

Figure 21: Server Deployment ports

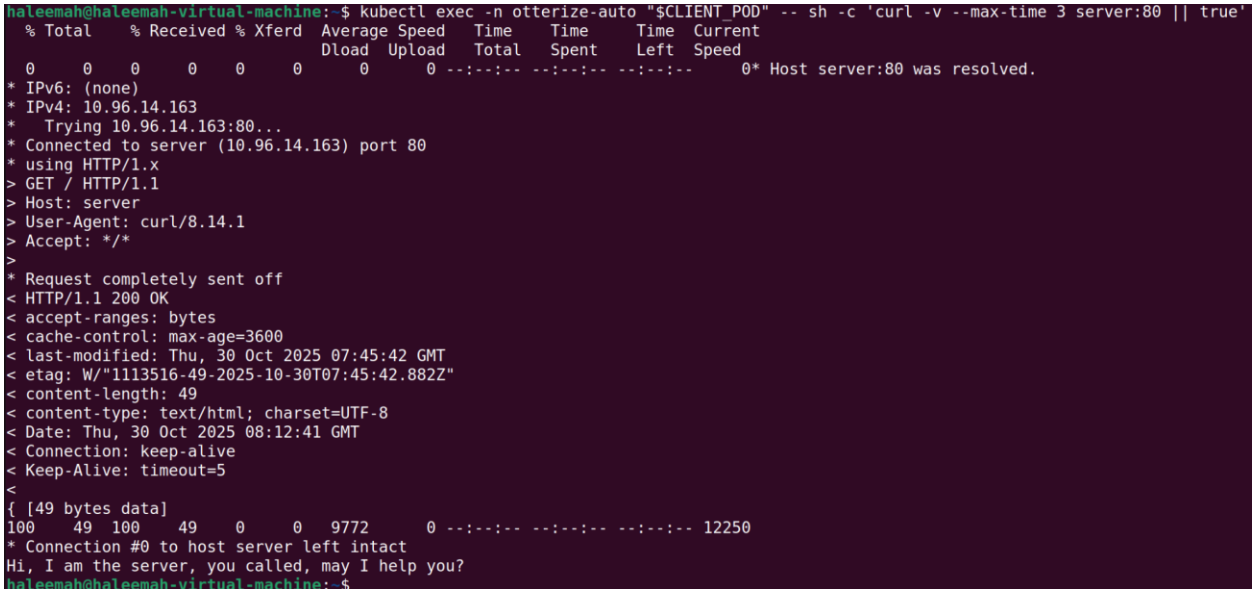


Figure 22: Client request to server using curl

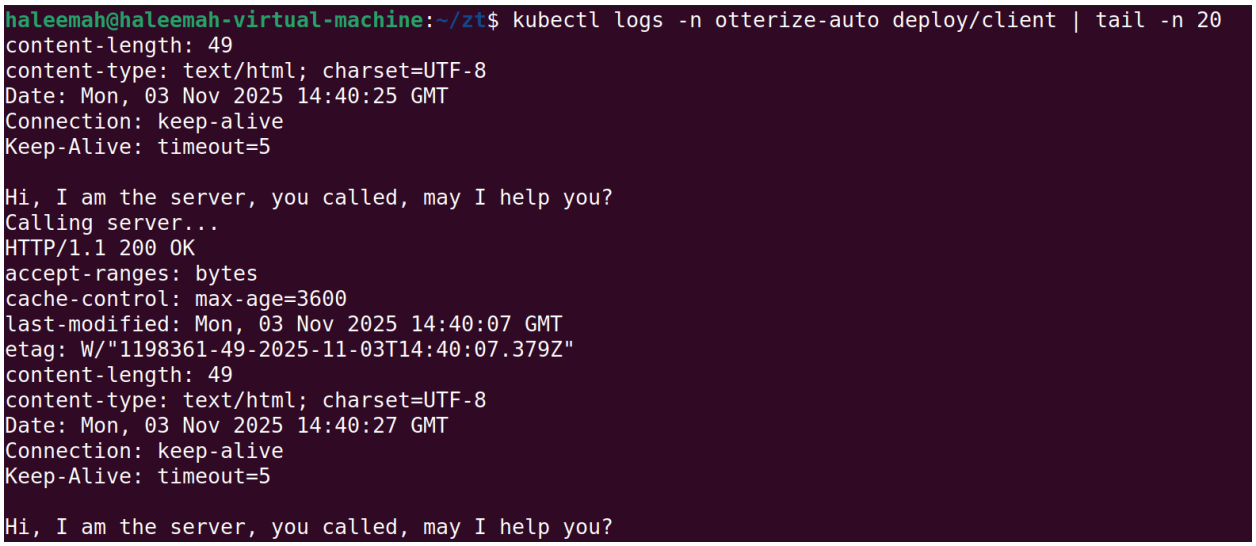


Figure 23: Client deployment logs

6.2 Single Run Script Results

The results of the single-run controller script were analyzed using multiple sources, including the controller job logs (Figure 22), the updated server deployment configuration (Figure 23), curl

test outputs (Figure 24), and server and client logs (Figures 25, 26). Collectively, these confirm that the script executed as intended and performed the following operations successfully:

- Retrieved the `ClientIntent` resources from the specified namespace.
- Extracted the **deny-port** value from the `ClientIntent` annotation.
- Identified the target deployment (workload) name and corresponding namespace.
- Fetched all containers defined within the target deployment.
- Patched the deployment by removing the specified port from each container's configuration.

These results demonstrate that the single-run controller effectively implemented the port denial logic and completed its execution after processing the existing intents.

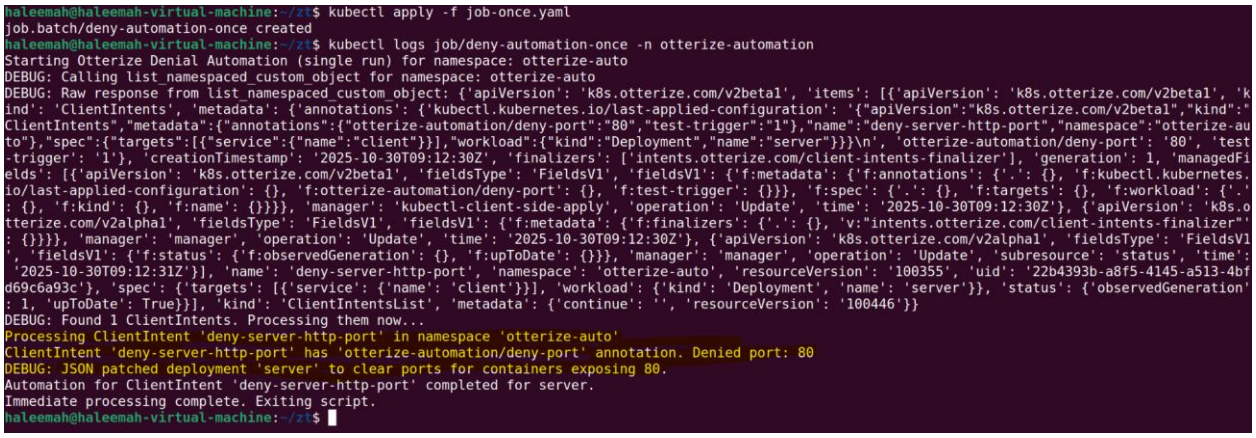


Figure 24: Single Run Script Controller(job) logs



Figure 25: Server deployment yaml config

```
haleemah@haleemah-virtual-machine:~/zt$ CLIENT_POD=$(kubectl get pod -n otterize-auto -l app=client -o jsonpath='{.items[0].metadata.name}')
haleemah@haleemah-virtual-machine:~/zt$ kubectl exec -n otterize-auto "$CLIENT_POD" -- sh -c 'curl -v --max-time 3 server:80 || true'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Host server:80 was resolved.
* IPv6: (none)
* IPv4: 10.96.14.163
*   Trying 10.96.14.163:80...
* connect to 10.96.14.163 port 80 from 10.244.0.7 port 53566 failed: Connection refused
* Failed to connect to server port 80 after 1 ms: Could not connect to server
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* closing connection #0
curl: (7) Failed to connect to server port 80 after 1 ms: Could not connect to server
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 26: Curl command output

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl logs -n otterize-auto deploy/server | tail -n 20

Starting up http-server, serving ./

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://10.244.0.13:8080
Hit CTRL-C to stop the server

haleemah@haleemah-virtual-machine:~/zt$
```

Figure 27: Server deployment logs

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl logs -n otterize-auto deploy/client | tail -n 20
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 27: Client deployment logs

6.3 Resetting Environment:

After completing the single-run script tests, the environment must be reset to its baseline state (Figure 27) before proceeding with further evaluations. This ensures a clean and consistent setup for subsequent testing of the continuous watch controller.

The reset script performs the following operations:

- Deletes any existing ClientIntent resources in the `otterize-auto` namespace.
- Removes the custom controller Job or Deployment, depending on which version was executed.
- Reapplies the original application resources and deployments using the `all.yaml` file.
- Restarts the server deployment to ensure it reverts to its baseline configuration.

Once the reset script completes, verification confirms that the server deployment has been restored to its baseline (pre-patch) state, and all ClientIntent resources and custom controller components have been successfully removed (Figure 28).

```
haleemah@haleemah-virtual-machine:~/zt$ ./reset_baseline.sh
[1/5] Remove any ClientIntents in test namespace (clears deny rules)
clientintents.k8s.otterize.com "deny-server-http-port" deleted from otterize-auto namespace
[2/5] Remove any Jobs from the namespace
job.batch "deny-automation-once" deleted from otterize-automation namespace
[3/5] Delete existing Deployment
[4/5] Re-apply original resources
namespace/otterize-auto unchanged
deployment.apps/client unchanged
deployment.apps/client-other unchanged
deployment.apps/server configured
service/server unchanged
[5/5] Restart server Deployment to pick up baseline spec
deployment.apps/server restarted
Waiting for deployment "server" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "server" rollout to finish: 1 old replicas are pending termination...
deployment "server" successfully rolled out
Baseline restored: server listening on port 80 and reachable from client.
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 28: reset_baseline script output

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl get deploy server -n otterize-auto -o yaml  | grep -A6 "ports:"
        ports:
        - containerPort: 80
          name: http
          protocol: TCP
        - containerPort: 8080
          name: http-alt
          protocol: TCP
haleemah@haleemah-virtual-machine:~/zt$ kubectl get pods -n otterize-automation
No resources found in otterize-automation namespace.
haleemah@haleemah-virtual-machine:~/zt$ kubectl get clientintents -n otterize-auto
No resources found in otterize-auto namespace.
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 29: Environment Baseline state (server port 80 exists, no controller deployments running, no client intents found)

6.4 Continuous Watch Script:

The Continuous Watch Controller extends the functionality of the single-run script by continuously monitoring the `ClientIntents` resource for real-time changes and reacting accordingly.

Analysis of the controller deployment logs (Figure 29), server YAML configuration (Figure 30), curl command outputs (Figure 31), and application logs for client pods (Figures 32) confirm that the Continuous Watch script operates as intended, performing actions based on the event type detected.

"ADDED" and "MODIFIED" Events: For these event types, the controller detects new or updated `ClientIntent` objects and attempts to remove the specified port from the target deployment.

- If the port exists, it is successfully removed, and the following confirmation message is logged.

```
"Automation for ClientIntent '{intent_name}' completed
for {workload_name}."
```

- If the specified port is not found, the controller handles the scenario gracefully and logs the message.

```
"Automation for ClientIntent '{intent_name}' failed to
remove port {denied_port}."
```

"DELETED" Event: When a `ClientIntent` is deleted, the controller is intentionally designed not to revert the changes previously applied. Instead, it logs a message indicating that no re-addition action was performed.

```
"ClientIntent '{intent_name}' was deleted. No action taken
for port re-addition based on current script logic"
```

```
haleemah@haleemah-virtual-machine:~/zt-2$ kubectl logs -f deployment/deny-automation-controller -n otterize-automation
Starting Otterize Denial Automation (continuous watch) for namespace: otterize-auto
DEBUG: Attempting to establish watch on ClientIntents in namespace otterize-auto (Attempt 1)...
```

Figure 30: Custom Controller watching ClientIntent resource for any change

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl get deploy server -n otterize-auto -o yaml | grep -A6 "ports:"
        ports:
        - containerPort: 8080
          name: http-alt
          protocol: TCP
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 31: server yaml configuration

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl exec -n otterize-auto "$CLIENT_POD" -- sh -c 'curl -v --max-time 3 server:80 || true'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Host server:80 was resolved.
* IPv6: (none)
* IPv4: 10.96.14.163
*   Trying 10.96.14.163:80...
* connect to 10.96.14.163 port 80 from 10.244.0.7 port 36964 failed: Connection refused
* Failed to connect to server port 80 after 1 ms: Could not connect to server
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* closing connection #0
curl: (7) Failed to connect to server port 80 after 1 ms: Could not connect to server
haleemah@haleemah-virtual-machine:~/zt$
```

Figure 32: client failing to reach server on port 80

```
haleemah@haleemah-virtual-machine:~/zt$ kubectl logs deploy/client -n otterize-auto | tail -n 20
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
Calling server...
curl timed out
```

Figure 33: Client logs

```
Processing event 'ADDED' for ClientIntent 'deny-server-http-port' in namespace 'otterize-auto'
ClientIntent 'deny-server-http-port' has 'otterize-automation/deny-port' annotation. Denied port: 80
DEBUG: JSON patched deployment 'server' to clear ports for containers exposing 80.
Automation for ClientIntent 'deny-server-http-port' completed for server.
```

Figure 34: Controller Detected “ADDED” Event

```
Processing event 'MODIFIED' for ClientIntent 'deny-server-http-port' in namespace 'otterize-auto'
ClientIntent 'deny-server-http-port' has 'otterize-automation/deny-port' annotation. Denied port: 80
DEBUG: No container exposed port 80. No patch built.
Automation for ClientIntent 'deny-server-http-port' failed to remove port 80.
```

Figure 35: Controller Detected “MODIFIED” Event

```
Processing event 'DELETED' for ClientIntent 'deny-server-http-port' in namespace 'otterize-auto'
ClientIntent 'deny-server-http-port' was deleted. No action taken for port re-addition based on current script logic.
```

Figure 36: Controller Detected “DELETED” Event

6.5 Application-Layer Attack Evaluation and Runtime Response

This section evaluates the effectiveness of the extended Zero Trust architecture by analyzing system behavior before and after the activation of the runtime traversal controller. The evaluation focuses on three aspects: baseline application accessibility, exploitability under a directory traversal attack, and system behavior following automated remediation.

6.5.1 Baseline System Behavior Before Controller Activation

Prior to deploying the traversal controller and enabling the intent-based enforcement, the system was configured with the ModSecurity Web Application Firewall acting as a reverse proxy. At this stage, no custom traversal rules were active, and the WAF operated in a permissive configuration.

External clients were able to access the backend application through the exposed service, and normal HTTP requests were successfully forwarded to the server. This confirmed that the WAF integration itself did not disrupt legitimate application functionality.



Figure 37: External client accessing the web application before controller activation.

6.5.2 Directory Traversal Attack Before Runtime Enforcement

To establish exploitability, a directory traversal attack was executed against the exposed web service before the traversal controller was running. The attack attempted to access sensitive system files by manipulating the request path.

The request was successfully forwarded through the WAF and reached the backend server, indicating that no traversal-specific protections were active at this stage. This confirms that, while the service was protected at the network level, application-layer inspection alone was insufficient without targeted enforcement.

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$  curl -v "http://172.18.255.200/../../../etc/passwd"
*   Trying 172.18.255.200:80...
* Connected to 172.18.255.200 (172.18.255.200) port 80 (#0)
> GET /etc/passwd HTTP/1.1
> Host: 172.18.255.200
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: nginx
< Date: Sun, 11 Jan 2026 00:38:48 GMT
< Content-Type: application/octet-stream
< Content-Length: 961
< Connection: keep-alive
< accept-ranges: bytes
< cache-control: max-age=3600
< last-modified: Tue, 23 May 2023 09:00:41 GMT
< etag: W/"1589115-961-2023-05-23T09:00:41.000Z"
< Access-Control-Allow-Headers: *
<
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
node:x:1000:1000::/home/node:/bin/bash
* Connection #0 to host 172.18.255.200 left intact
```

Figure 38: Successful directory traversal attempt before runtime enforcement

6.5.3 Controller Deployment and Intent Activation

Following the baseline evaluation, the traversal controller container image was loaded into the Kubernetes cluster and deployed within the automation namespace. An Otterize ClientIntent was then applied with an explicit annotation enabling traversal remediation.

This intent acted as a feature flag, authorizing the controller to perform enforcement actions when an attack was detected. At this point, no rules had yet been injected, and the system behavior remained unchanged until a traversal attempt occurred.

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$ cat enable-modsec-traversal-intent.yaml
apiVersion: k8s.otterize.com/v2beta1
kind: ClientIntents
metadata:
  name: enable-modsec-traversal
  namespace: otterize-auto
  annotations:
    otterize-automation/enable-modsec-crs-traversal: "true"
spec:
  workload:
    name: server
    kind: Deployment
```

Figure 39: client intents file

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$ kubectl apply -f enable-modsec-traversal-intent.yaml
clientintents.k8s.otterize.com/enable-modsec-traversal created
```

Figure 40: apply the intent command

6.5.4 Runtime Detection and Automated Remediation

After the controller was running, a directory traversal request was executed again. The controller continuously monitored server pod logs and detected the traversal pattern within recent log entries. Upon detection, the controller verified that the intent-based authorization was enabled.

Once authorized, the controller dynamically appended a custom traversal-blocking rule to the ModSecurity ConfigMap and triggered a controlled restart of the WAF deployment. Controller logs confirm each stage of this process, including detection, authorization, rule injection, and restart initiation.

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$ kubectl logs -f deployment/traversal-controller -n otterize-automation
INFO: Loaded in-cluster Kubernetes config.
INFO: Starting server log monitor for directory traversal detection...
ALERT: Directory traversal pattern detected in logs of pod 'server-764f5576f7-wfnzv'.
INFO: Feature flag otterize-automation/enable-modsec-crs-traversal = true (enabled=True)
INFO: Patched ConfigMap modsec-custom-rules to add traversal rule id=100010.
INFO: Patched Deployment modsecurity-waf to trigger restart (traversal-rule-ts=1768092671).
INFO: WAF deployment restart triggered to load updated traversal rule.
ALERT: Directory traversal pattern detected in logs of pod 'server-764f5576f7-wfnzv'.
INFO: Traversal rule already enabled earlier; no further action.
```

Figure 41: Traversal controller logs showing detection and automated rule enforcement

6.5.5 Post-Remediation Behavior

After remediation, the same directory traversal request was attempted again. The WAF blocked the request and returned an HTTP 403 response, indicating that the custom ModSecurity rule was active and functioning correctly.

Importantly, normal application requests continued to be forwarded successfully through the WAF. This confirms that enforcement was selective and did not disrupt legitimate traffic, unlike the earlier port removal experiment.

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$ curl -v "http://172.18.255.200/../../../etc/passwd"
*   Trying 172.18.255.200:80...
* Connected to 172.18.255.200 (172.18.255.200) port 80 (#0)
> GET /etc/passwd HTTP/1.1
> Host: 172.18.255.200
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Sun, 11 Jan 2026 00:51:23 GMT
< Content-Type: text/plain
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host 172.18.255.200 left intact
```

Figure 42: Directory traversal attempt blocked after runtime remediation

```
haleemah@haleemah-VirtualBox:~/Downloads/files/zt-2$ curl -v http://172.18.255.200/
*   Trying 172.18.255.200:80...
* Connected to 172.18.255.200 (172.18.255.200) port 80 (#0)
> GET / HTTP/1.1
> Host: 172.18.255.200
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: nginx
< Date: Sun, 11 Jan 2026 01:05:24 GMT
< Content-Type: text/html; charset=UTF-8
< Content-Length: 49
< Connection: keep-alive
< accept-ranges: bytes
< cache-control: max-age=3600
< last-modified: Sun, 11 Jan 2026 00:31:53 GMT
< etag: W/"1349255-49-2026-01-11T00:31:53.818Z"
< Access-Control-Allow-Headers: *
<
Hi, I am the server, you called, may I help you?
* Connection #0 to host 172.18.255.200 left intact
```

Figure 43: Normal application traffic continues after traversal rule activation

6.5.6 Evaluation Summary

The evaluation results demonstrate that the extended Zero Trust framework successfully detects and mitigates an application-layer attack at runtime. Before enforcement, the system was vulnerable to directory traversal despite network-level protections. After controller activation

and intent authorization, malicious requests were blocked while normal application behavior was preserved.

This confirms that integrating application-layer inspection with intent-driven runtime automation improves system resilience without sacrificing availability.

7. Critical Analysis

7.1 Strengths and Weaknesses of the Proposed Solution

7.1.1 Strengths

The proposed zero-trust solution for Kubernetes demonstrates a robust security architecture that tightly integrates runtime monitoring with policy enforcement and automated remediation. Its design is modular and built on Kubernetes-native principles. A custom controller reacts to security intents (via Otterize's *ClientIntents* CRDs) and patches deployments on the fly. This modular approach leverages existing platforms (Otterize) rather than reinventing the wheel, reflecting flexibility and ease of integration into the cloud-native ecosystem. The ease of implementation is evidenced by the use of standard Kubernetes APIs (e.g. a PATCH to the Deployment) to enact changes, meaning the solution can be deployed without complex modifications to the cluster. In practice, the authors report that their integrated approach significantly strengthened the cluster's security posture, enabling rapid threat mitigation and visibly reducing the application attack surface at runtime.

7.1.2 Weaknesses

Despite its merits, the solution has clear limitations in threat coverage and automation. Some of the weaknesses as listed below are:

- **Detection is not entirely autonomous.** The runtime "detection" loop is triggered by declarative security intents authored out-of-band; the architecture does not integrate a native runtime signal source (e.g., syscall-, network-, or behavior-based detectors) that can generate those intents automatically. This leaves a human- or tool-in-the-loop gap before response can begin.

- **Narrow threat-model coverage.** The design primarily operationalizes least-privilege at the service/port boundary. It does not specify controls for process integrity, file/secret abuse, in-container privilege escalation, lateral movement, or supply-chain indicators, reducing effectiveness against non-network manifestations of runtime compromise.
- **Change-safety and rollback semantics.** There is no explicit mechanism for safe rollout (dry-run, canary, backoff), automatic rollback, or partial-failure isolation when enforcement actions misfire. Absent these guardrails, remediation can cause self-inflicted outages.
- **Controller trust and hardening.** The controller requires privileges to watch intents and patch workloads. The architecture does not discuss hardening (minimal RBAC, admission constraints, namespace scoping, signing of images) or resilience to controller compromise, creating a high-value target.

7.2 Reproducibility, Scalability, and Practicality of the solution

7.2.1 Reproducibility: Adaptability to Local Environments

A practical strength of the original framework is that its main architecture is not dependent on a managed cloud Kubernetes service. Although Amadoru (2025) implemented the original Proof of Concept using Google Kubernetes Engine (GKE), the framework relies primarily on Kubernetes resources, a custom controller, and Otterize ClientIntents. This made it possible to reproduce the main architecture within the local Kind-based Kubernetes environment used in this study.

The local implementation also supported the modifications introduced in the proposed framework, including application-log monitoring, ClientIntent-based authorization, dynamic updates to the ModSecurity ConfigMap, and rolling restarts of the WAF Deployment. These functions were implemented without relying on GKE-specific services. This demonstrates that the framework can be reproduced and extended in a local Kubernetes environment while retaining the automated detection, authorization, and remediation workflow.

However, the successful local implementation should not be interpreted as evidence that the framework will behave identically in production or large-scale Kubernetes environments. The

evaluation was conducted within a controlled local environment and did not examine factors such as multi-node deployment, high traffic volumes, autoscaling, or cloud-specific networking. The results therefore support the reproducibility and portability of the Proof of Concept architecture, rather than establishing production-level scalability.

7.2.2 Practicality of the Approach

From a practical deployment perspective, the original approach demonstrates that automated remediation can be integrated with Kubernetes policy enforcement, but its response mechanism is designed around a specific security condition. In the automated port-denial scenario, the controller responds to an exposed port by patching the affected Deployment and removing that port. While this demonstrates the feasibility of automated enforcement, a broader runtime security framework would need to support different types of threats and allow the response to be selected according to the behaviour being detected. Different threats may require different actions, such as restricting a specific request, isolating a workload, or modifying an access policy, rather than applying the same remediation in every situation.

Another practical limitation is that the original framework does not include an integrated mechanism for detecting malicious runtime behaviour. The Custom Security Controller reacts to changes in ClientIntent resources containing the configured deny-port condition, meaning that the security condition must already be represented through the policy resource before remediation begins. As a result, the framework automates the enforcement response but does not itself identify the malicious activity that should trigger that response. This creates a separation between runtime threat detection and automated remediation.

The proposed framework addresses this limitation within the scope of the directory traversal scenario by introducing application logs as a source of runtime evidence. The Custom Security Controller monitors these logs for directory traversal patterns and, when matching activity is identified, evaluates the corresponding ClientIntent to determine whether remediation is authorized. If authorized, the controller updates the ModSecurity ConfigMap and triggers the WAF Deployment to load the new rule. This connects runtime detection and automated response within the same workflow while preserving policy-based authorization.

However, the proposed framework remains limited to a specific attack type and detection pattern. Supporting a wider range of runtime threats would require additional detection logic and appropriate response mechanisms for each threat. The implementation therefore demonstrates how the detection-to-response gap in the original framework can be addressed for directory traversal attacks, rather than providing a general-purpose automated response system.

7.2.3 Scalability

Scalability remains an open question because neither the original study nor the evaluation conducted in this project tested the framework in a large-scale Kubernetes environment. The Proof of Concept was evaluated within a limited cluster, so there is insufficient evidence to determine how the Custom Security Controller would perform across a larger environment containing many nodes, Pods, Deployments, and simultaneous security events. Factors such as controller CPU and memory usage, response time, and the number of interactions with the Kubernetes API were not formally evaluated.

The proposed framework introduces additional scalability considerations because the Custom Security Controller periodically retrieves and analyzes application logs for evidence of directory traversal activity. In a larger environment containing many application Pods or high volumes of HTTP traffic, the amount of log data that must be processed could increase considerably. Multiple detections occurring within a short period could also result in repeated ClientIntent evaluations, ModSecurity ConfigMap updates, and WAF Deployment restarts. The current evaluation did not measure how these operations would behave under sustained or concurrent attack activity.

For these reasons, the results of this study demonstrate the feasibility of the proposed workflow within a controlled local Kubernetes environment rather than its scalability for production deployment. Further evaluation would be required to measure controller resource consumption, detection and remediation latency, Kubernetes API load, behaviour under concurrent security events, and the effect of repeated WAF configuration updates. Testing these factors across larger multi-node clusters would provide stronger evidence of whether the framework can maintain reliable automated enforcement as workload and traffic volume increase.

7.3 Critical Analysis of the Application-Layer Runtime Extension

This section critically examines the application-layer extension introduced in the proposed framework. The analysis considers how targeted ModSecurity enforcement changes the response implemented in the original framework, with particular attention to enforcement granularity, application availability, and the limitations introduced by request-level protection.

7.3.1 Effectiveness of Runtime Application-Layer Enforcement

The results show that the application-layer extension successfully contained the tested directory traversal attack while preserving legitimate access to the application. The Custom Security Controller identified the directory traversal pattern from application logs, evaluated remediation authorization through the ClientIntent, and automatically applied the targeted ModSecurity rule. After remediation, the malicious request was blocked with an HTTP 403 response, while legitimate application traffic continued to receive an HTTP 200 OK response.

This result addresses an important limitation of the original framework. Amadoru's (2025) remediation removes the exposed container port, preventing communication through that access path entirely. In the proposed framework, enforcement occurs at the request level instead. Rather than removing the application's communication path, ModSecurity evaluates incoming HTTP requests and blocks those matching the configured directory traversal rule. The experiment therefore demonstrates that automated runtime remediation can be applied more selectively while maintaining application availability for the legitimate traffic tested.

The framework also separates detection from authorization. Identifying a directory traversal pattern does not immediately result in a change to the security configuration; the ClientIntent must first authorize the remediation. This maintains policy-based control over the automated response and reduces the possibility of a detected event automatically triggering an unauthorized enforcement action.

However, the effectiveness of the approach depends on the detection pattern and ModSecurity rule used to identify malicious requests. In this experiment, detection and enforcement were designed specifically for directory traversal patterns. Requests that do not match the configured patterns may therefore avoid detection or enforcement. The results demonstrate the effectiveness

of the approach against the tested attack scenario, but they do not establish its effectiveness against other web attacks or more complex variations of directory traversal.

7.3.2 Operational Tradeoffs and Limitations

Despite the more targeted enforcement provided by the proposed framework, the approach introduces several operational trade-offs. First, the Custom Security Controller relies on application logs for detection. As a result, a malicious request must reach the application and be recorded before the controller can identify the directory traversal pattern and initiate remediation. This creates a short exposure window in which the initial request is not yet blocked by the dynamically generated ModSecurity rule.

A second limitation is that activating a new ModSecurity rule requires a rolling restart of the WAF Deployment. After the ModSecurity ConfigMap is updated, the controller patches the WAF Deployment so that the WAF Pod restarts and loads the new configuration. This approach was sufficient for the local Proof of Concept, but the study did not evaluate the performance or availability effects of repeated restarts under larger workloads. A rule-reloading mechanism that does not require restarting the WAF Pod could therefore be investigated in future work.

The framework also depends on correct ClientIntent configuration. When directory traversal activity is detected, remediation occurs only if the corresponding ClientIntent authorizes the response. If remediation is not authorized, the controller records the detection but does not modify the ModSecurity configuration. This provides explicit policy control over automated enforcement, but it also means that an incorrect or outdated ClientIntent configuration could prevent remediation even when malicious activity has been detected.

7.3.4 Implications for Zero Trust Resilience

The results of the extended experiment show how runtime evidence, policy-based authorization, and automated enforcement can be combined to support a more adaptive form of Zero Trust enforcement. Rather than relying only on access decisions established before an attack occurs, the proposed framework uses application-level evidence observed during runtime to determine when a security response may be required. The ClientIntent then provides an authorization step before the response is automatically applied.

The application-layer extension also demonstrates the value of applying enforcement at a more granular level. In the original framework, remediation removes the application's exposed container port, preventing communication through that path. The proposed framework instead uses ModSecurity to block requests matching the identified directory traversal pattern while allowing legitimate requests to continue. In the tested scenario, this allowed the framework to contain the malicious request without making the application unavailable to legitimate traffic.

These findings suggest that Zero Trust runtime enforcement can consider not only whether a response is automatically applied, but also the scope of that response. More targeted enforcement can reduce unnecessary restrictions on legitimate access while still responding to identified malicious behaviour. However, because the experiment was limited to a directory traversal attack within a local Kubernetes environment, further evaluation would be required to determine whether the same approach remains effective across other attack types and larger or more complex deployments.

8. Conclusion and Future Work

8.1 Conclusion

This study investigated whether the automated Zero Trust remediation approach proposed by Amadoru (2025) could be extended to provide more targeted runtime threat containment while preserving legitimate application access. The original framework was first reproduced within a local Kubernetes environment, demonstrating that its automated controller-based architecture could operate without relying on the original Google Kubernetes Engine environment.

The proposed framework then extended this architecture by using application logs as runtime evidence of directory traversal activity and ModSecurity as the application-layer enforcement mechanism. The Custom Security Controller monitored application logs for the configured traversal pattern, evaluated remediation authorization through the ClientIntent, updated the ModSecurity ConfigMap, and triggered the WAF Deployment to load the new security rule. This created an automated workflow connecting runtime detection, policy-based authorization, and targeted enforcement.

The evaluation showed that the proposed approach successfully contained the tested directory traversal attack while preserving legitimate application access. After remediation, the malicious request was blocked with an HTTP 403 response, while legitimate traffic continued to receive an HTTP 200 OK response. Unlike the original remediation approach, which removes the exposed container port and prevents communication through that path, the modified framework demonstrated that enforcement could be applied selectively at the request level without making the application unavailable to the legitimate traffic tested.

The results therefore demonstrate the feasibility of connecting application-level runtime evidence with automated, policy-controlled enforcement in a local Kubernetes environment. However, the findings are limited to the tested directory traversal scenario and local experimental environment. The framework's dependence on predefined detection patterns, application logs, and WAF restarts also introduces limitations that require further investigation. The results should therefore be understood as evidence of the feasibility of the proposed approach rather than proof of its effectiveness across all attack types or production-scale Kubernetes environments.

8.2 Future Work

- **Expanding Runtime Detection:** Future work could extend the current pattern-based detection mechanism to identify a wider range of malicious behaviour. The present framework was evaluated using predefined directory traversal patterns identified through application logs. Additional detection techniques could be investigated to determine whether the same automated response architecture can support other web attacks or more complex variations that may not match known patterns.
- **Improving Dynamic Rule Activation:** The current implementation requires a rolling restart of the WAF Deployment after the ModSecurity ConfigMap is updated. Future work could investigate mechanisms for dynamically reloading ModSecurity rules without restarting the WAF Pod. This could reduce the operational overhead associated with repeated remediation events and shorten the time between detection and active enforcement.
- **Integrating Rollback Safeguards:** Future versions of the framework could introduce rollback mechanisms for automatically applied ModSecurity rules. If a dynamically

generated rule incorrectly blocks legitimate traffic, the framework could remove or revert the rule based on predefined conditions. This would provide an additional safeguard against unintended effects from automated remediation.

- **Enhancing Controller Security:** Although the current implementation restricts the Custom Security Controller through RBAC and a dedicated service account, additional security measures could be investigated. These could include stronger namespace isolation, container image verification, controller health monitoring, and additional auditing of automated changes.
- **Evaluating Additional Attack Scenarios:** The current evaluation focuses on directory traversal. Future testing could examine whether the same workflow of runtime detection, ClientIntent authorization, and targeted application-layer enforcement can be adapted to other web application attacks. This would help determine how generalizable the proposed framework is beyond the attack scenario evaluated in this study.
- **Validating Performance and Scalability:** The framework was evaluated within a local Kubernetes environment and was not tested under production-scale workloads. Future research could evaluate larger or multi-node Kubernetes clusters, higher request volumes, and repeated security events to measure detection time, remediation latency, application availability, and the operational impact of automated enforcement.

References

Aly, A., Hamad, A. M., Al-Qutt, M., & Fayez, M. (2025). Real-time multi-class threat detection and adaptive deception in Kubernetes environments. *Scientific Reports, 15*, Article 8924. https://doi.org/10.1038/s41598-025-91606-8

Amadoru, N. U. (2025). *Enhancing zero-trust security in Kubernetes: Bridging the gap in runtime threat detection and automated response* [Master's thesis, Aalto University]. Aalto University. https://urn.fi/URN:NBN:fi:aalto-202508196295

Bagheri, S., Kermabon-Bobinnec, H., Kabir, M. E., Majumdar, S., Wang, L., Jarraya, Y., Nour, B., & Pourzandi, M. (2024). ACE-WARP: A cost-effective approach to proactive and non-disruptive incident response in Kubernetes clusters. *IEEE Transactions on Information Forensics and Security, 19*, 8204–8219. https://doi.org/10.1109/TIFS.2024.3449038

Cao, C., Blaise, A., Verwer, S., & Rebecchi, F. (2022). Learning state machines to monitor and detect anomalies on a Kubernetes cluster. In *Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022)*. Association for Computing Machinery. https://doi.org/10.1145/3538969.3543810

Darwesh, G., Hammoud, J., & Vorobeva, A. A. (2022). Security in Kubernetes: Best practices and security analysis. *Journal of the Ural Federal District: Information Security, 2*(44), 63–69. https://doi.org/10.14529/secur220209

Díaz-Verdejo, J., Muñoz-Calle, J., Estepa Alonso, R., & Estepa Alonso, A. (2024). InspectorLog: A new tool for offline attack detection over web log trace files. In *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)* (pp. 692–697). SciTePress. https://doi.org/10.5220/0012764000003767

Joshi, A., & Segireddy, A. (2026). Adaptive runtime security for Kubernetes using eBPF telemetry and machine learning. In *Proceedings of the 6th International Conference on Expert Clouds and Applications (ICOECA 2026)* (pp. 1663–1668). IEEE. https://doi.org/10.1109/ICOECA68095.2026.11485567

MK, A., Bala, K. S. S., Sonti, S. S. T., & KP, J. (2026). An empirical study on the evaluation and enhancement of OWASP CRS (Core Rule Set) in ModSecurity. *Computers & Security, 160*, Article 104714. https://doi.org/10.1016/j.cose.2025.104714

Red Hat. (2024). *The state of Kubernetes security report: 2024 edition*. https://www.redhat.com/en/engage/state-kubernetes-security-report-2024

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). *Zero trust architecture* (NIST Special Publication 800-207). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207

Shenoy, A. M., Swetha, P., & Honnavali, P. B. (2025). Runtime threat mitigation in Kubernetes using Falco, Falcosidekick, and Kyverno: An automated pod deletion approach. In *2025 IEEE 7th International Conference on Computing, Communication and Automation (ICCCA)*. IEEE. https://doi.org/10.1109/ICCCA66364.2025.11325363

A. Appendix

A.1 Test Application

A.1.1 Application Workload Manifests

```
   # Defines Kubernetes workloads (client, client-other, server) and
a service for the automated port-denial Zero-Trust experiment.

apiVersion: v1
kind: Namespace
metadata:
  name: otterize-auto
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client
  namespace: otterize-auto
spec:
  selector:
    matchLabels:
      app: client
  template:
    metadata:
      labels:
        app: client
    spec:
      containers:
        - name: client
          image: alpine/curl
          command: ["/bin/sh", "-c", "--"]
          args:
            [
              'while true; do echo "Calling server..."; if ! timeout
2 curl -si server 2>/dev/null; then echo "curl timed out"; fi; sleep
2; done',
            ]
---
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-other
  namespace: otterize-auto
spec:
  selector:
    matchLabels:
      app: client-other
  template:
    metadata:
      labels:
        app: client-other
    spec:
      containers:
        - name: client-other
          image: alpine/curl
          command: ["/bin/sh", "-c", "--"]
          args:
            [
              'while true; do echo "Calling server..."; if ! timeout
2 curl -si server 2>/dev/null; then echo "curl timed out"; fi; sleep
2; done',
            ]

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: server
  namespace: otterize-auto
spec:
  selector:
    matchLabels:
      app: server
  template:
    metadata:
      labels:
        app: server
```

```
    spec:
      containers:
        - name: server
          image: node:19
          command: ["/bin/sh", "-c"]
          args:
            [
              'echo "Hi, I am the server, you called, may I help
you?" > index.html; npx --yes http-server -p 80 -a 0.0.0.0 & npx --
yes http-server -p 8080 -a 0.0.0.0',
            ]
          ports:
            - containerPort: 80
              protocol: TCP
              name: http
            - containerPort: 8080
              protocol: TCP
              name: http-alt
---
apiVersion: v1
kind: Service
metadata:
  name: server
  namespace: otterize-auto
spec:
  selector:
    app: server
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: http # matches containerPort name
    - name: http-alt
      protocol: TCP
      port: 8080
      targetPort: http-alt # matches containerPort name
```

A.1.2 ClientIntent Manifest:

Same as original thesis

```
   # Declares an Otterize ClientIntent resource that triggers the
custom controller to deny a specific application port.
apiVersion: k8s.otterize.com/v2beta1
kind: ClientIntents
metadata:
  name: deny-server-http-port
  namespace: otterize-auto
  annotations:
    otterize-automation/deny-port: "80"
    test-trigger: "1"
spec:
  workload:
    name: server
    kind: Deployment
  targets:
    - service:
        name: client




   #Clientintent experiment #2

    apiVersion: k8s.otterize.com/v2beta1

   kind: ClientIntents

   metadata:

     name: enable-modsec-traversal

     namespace: otterize-auto

     annotations:

       otterize-automation/enable-modsec-crs-traversal: "true"

   spec:
```

```
    workload:
      name: server
      kind: Deployment
    targets: []
```

A.1.3 Dockerization

Same as original thesis

```
   # Dockerfile for creating Custom Controller image

# Python3 base image
FROM python:3.9-slim-buster

# Set working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install needed packages specified in requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copy the script into the container
COPY deny_automation.py .

# Command to run the script when the container starts
# The script will read NAMESPACE_TO_WATCH from an environment
variable
CMD ["python", "deny_automation.py"]
```

Requirements file(same as original thesis)

```
   kubernetes>=28.1.0
```

A.1.4 Role Based Access Control Rules

Same as original thesis, but not provided in the appendix section of the theses. Instead they provided Screen shots of each section of the file.

```
   # Configures namespaces, ServiceAccount, Role, and RoleBinding for
granting the deny-automation controller required permissions.

apiVersion: v1
kind: ServiceAccount
metadata:
  name: deny-automation-controller
  namespace: otterize-automation
  labels:
    app.kubernetes.io/name: deny-automation-controller
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deny-automation-controller-role
  namespace: otterize-auto
  labels:
    app.kubernetes.io/name: deny-automation-controller
rules:
  # Read Otterize ClientIntents CRs in otterize-auto
  - apiGroups: ["k8s.otterize.com"]
    resources: ["clientintents"]
    verbs: ["get", "list", "watch"]

  # Read/modify Deployments in otterize-auto
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "update", "patch"]
```

```
  # Optional diagnostics in otterize-auto
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deny-automation-controller-rolebinding
  namespace: otterize-auto
  labels:
    app.kubernetes.io/name: deny-automation-controller
subjects:
  - kind: ServiceAccount
    name: deny-automation-controller
    namespace: otterize-automation   # SA lives where the controller
runs
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: deny-automation-controller-role
```

A.2 Automation Code

A.2.1 Imports and Environment variables:

```
   import sys, time, os
from kubernetes import client, config, watch
from kubernetes.client import ApiException

NAMESPACE_TO_WATCH = os.getenv("NAMESPACE_TO_WATCH", "otterize-auto")
DENY_PORT_ANNOTATION = "otterize-automation/deny-port"
```

A.2.2 Helper Functions

_init_kube_clients: Helper function for initializing Kubernetes client.

```
   def _init_kube_clients():
    # Try in-cluster first, fall back to local kubeconfig
    try:
        config.load_incluster_config()
    except Exception:
        config.load_kube_config()
    return client.CustomObjectsApi(), client.AppsV1Api()
```

get_denied_port_from_annotation: Helper function for Extracting port number from annotation

```
   def get_denied_port_from_annotation(obj: dict):
    """
    Return the denied port from ClientIntents annotations, or None if
missing/invalid.
    Expects DENY_PORT_ANNOTATION to be defined.
    """
    annotations = (obj.get("metadata") or {}).get("annotations") or
{}
    raw = annotations.get(DENY_PORT_ANNOTATION)
    if raw is None:
        return None

    try:
        port = int(str(raw).strip())
    except (ValueError, TypeError):
        print(f"Warning: '{DENY_PORT_ANNOTATION}' value is not an
integer: {raw}")
        sys.stdout.flush()
        return None

    if not (1 <= port <= 65535):
        print(f"Warning: '{DENY_PORT_ANNOTATION}' out of range 1-
65535: {port}")
        sys.stdout.flush()
```

```
        return None

    return port
```

Remove_port_from_deployment: helper function for patching deployment

```
   def remove_port_from_deployment(namespace, deployment_name,
denied_port):
    try:
        dep =
apps_v1.read_namespaced_deployment(name=deployment_name,
namespace=namespace)
    except ApiException as e:
        if e.status in (403, 404):
            print("Hint: verify RBAC and that the Deployment
exists.")
        print(f"Error: read_namespaced_deployment failed: {e}");
sys.stdout.flush()
        return False

    d = dep.to_dict()
    containers = (d.get("spec", {}).get("template", {}).get("spec",
{}).get("containers") or [])
    if not containers:
        print("DEBUG: Deployment has no containers.");
sys.stdout.flush()
        return False

    ops = []
    # build JSON Patch ops: wipe ports [] for any container that
exposes denied_port
    for idx, c in enumerate(containers):
        ports = c.get("ports") or []
        # find indices of entries whose container_port/containerPort
== denied_port
        rm_idxs = []
        for j, p in enumerate(ports):
```

```
            raw = p.get("container_port", p.get("containerPort"))
            try:
                cp = int(raw) if raw is not None else None
            except Exception:
                cp = None
            if cp == int(denied_port):
                rm_idxs.append(j)

        # JSON Patch: remove matching items from the list
        # reverse is necessary because if we remove index 0 first
from port array, then later indexes will shift down and not match our
rm_idxs array
        for j in sorted(rm_idxs, reverse=True):
            ops.append({
                "op": "remove",
                "path":
f"/spec/template/spec/containers/{idx}/ports/{j}"
            })

    if not ops:
        print(f"DEBUG: No container exposed port {denied_port}. No
patch built."); sys.stdout.flush()
        return False

    # ensure annotations map exists, then add/replace a timestamp to
force new RS
    ops.append({
        "op": "add",
        "path": "/spec/template/metadata/annotations",
        "value": d.get("spec", {}).get("template",
{}).get("metadata", {}).get("annotations", {}) or {}
    })
    # escape "/" in JSON Patch paths with "~1"
    ops.append({
        "op": "add",  # add acts like upsert for a missing key
        "path": "/spec/template/metadata/annotations/deny-auto~1ts",
        "value": str(int(time.time()))
    })
```

```
    # apply with JSON Patch content-type; retry lightly on conflicts
    for delay in (0, 0.2, 0.5):
        try:
            apps_v1.patch_namespaced_deployment(
                name=deployment_name,
                namespace=namespace,
                body=ops,

            )
            print(f"DEBUG: JSON patched deployment
'{deployment_name}' to clear ports for containers exposing
{denied_port}.")
            sys.stdout.flush()
            return True
        except ApiException as e:
            if e.status == 409:
                time.sleep(delay); continue
            print(f"Error: JSON patch failed: {e}");
sys.stdout.flush()
            return False

    return False
```

A.2.3 Single Run Automation

Main function for single run version of the custom controller.

```
   # Python script for running the Single run version of the Custom
Controller
import sys, time, os
from kubernetes import client, config
from kubernetes.client import ApiException

def run_automation_once():
    print(f"Starting Otterize Denial Automation (single run) for
namespace: {NAMESPACE_TO_WATCH}")
    sys.stdout.flush()
```

```
    try:
        print(f"DEBUG: Calling list_namespaced_custom_object for
namespace: {NAMESPACE_TO_WATCH}")
        sys.stdout.flush()
        client_intents =
custom_objects_api.list_namespaced_custom_object(
            group="k8s.otterize.com",
            version="v2beta1",
            namespace=NAMESPACE_TO_WATCH,
            plural="clientintents"
        )
        print(f"DEBUG: Raw response from
list_namespaced_custom_object: {client_intents}")
        sys.stdout.flush()

        if 'items' not in client_intents or not
client_intents['items']:
            print("DEBUG: No ClientIntents found in the response
'items' list. Skipping processing loop.")
            sys.stdout.flush()
        else:
            print(f"DEBUG: Found {len(client_intents['items'])}
ClientIntents. Processing them now...")
            sys.stdout.flush()
            for obj in client_intents['items']:
                intent_name = obj['metadata']['name']
                intent_namespace = obj['metadata']['namespace']

                print(f"Processing ClientIntent '{intent_name}' in
namespace '{intent_namespace}'")
                sys.stdout.flush()

                denied_port = get_denied_port_from_annotation(obj)
                if denied_port:
                    print(f"ClientIntent '{intent_name}' has
'{DENY_PORT_ANNOTATION}' annotation. Denied port: {denied_port}")
                    sys.stdout.flush()
```

```
                    workload_name = None
                    if 'spec' in obj and 'workload' in obj['spec']
and 'name' in obj['spec']['workload']:
                        workload_name =
obj['spec']['workload']['name']

                    if not workload_name:
                        print(f"Warning: ClientIntent '{intent_name}'
does not specify a workload name. Skipping.")
                        sys.stdout.flush()
                        continue

                    try:

apps_v1.read_namespaced_deployment(workload_name, intent_namespace)
                    except ApiException as e:
                        if e.status == 404:
                            print(f"Warning: Deployment
'{workload_name}' not found for ClientIntent '{intent_name}'.
Skipping patch.")
                            sys.stdout.flush()
                            continue
                        else:
                            print(f"Error reading deployment
'{workload_name}': {e}")
                            sys.stdout.flush()
                            continue

                    success =
remove_port_from_deployment(intent_namespace, workload_name,
denied_port)
                    if success:
                        print(f"Automation for ClientIntent
'{intent_name}' completed for {workload_name}.")
                        sys.stdout.flush()
                    else:
                        print(f"Automation for ClientIntent
'{intent_name}' failed to remove port {denied_port}.")
```

```
                        sys.stdout.flush()
                else:
                    print(f"ClientIntent '{intent_name}' does not
have the '{DENY_PORT_ANNOTATION}' annotation or it's malformed.")
                    sys.stdout.flush()

    except Exception as e:
        print(f"Error fetching ClientIntents: {e}")
        sys.stdout.flush()

    print("Immediate processing complete. Exiting script.")
    sys.stdout.flush()
```

A.2.4 Continuous Deny Automation

Main function for continuous watch version of the custom controller

```
# Python script for running the Continuous watch version of the
Custom Controller. This is the complete/finsihed version of Custom
Controller
import sys, time, os
from kubernetes import client, config, watch
from kubernetes.client import ApiException



NAMESPACE_TO_WATCH = os.getenv("NAMESPACE_TO_WATCH", "otterize-auto")
DENY_PORT_ANNOTATION = "otterize-automation/deny-port"

def _init_kube_clients():
    # Try in-cluster first, fall back to local kubeconfig
    try:
        config.load_incluster_config()
    except Exception:
        config.load_kube_config()
    return client.CustomObjectsApi(), client.AppsV1Api()
```

```
# initialize kube clients
custom_objects_api, apps_v1 = _init_kube_clients()

def get_denied_port_from_annotation(obj: dict):
    """
    Return the denied port from ClientIntents annotations, or None if
missing/invalid.
    Expects DENY_PORT_ANNOTATION to be defined.
    """
    annotations = (obj.get("metadata") or {}).get("annotations") or
{}
    raw = annotations.get(DENY_PORT_ANNOTATION)
    if raw is None:
        return None

    try:
        port = int(str(raw).strip())
    except (ValueError, TypeError):
        print(f"Warning: '{DENY_PORT_ANNOTATION}' value is not an
integer: {raw}")
        sys.stdout.flush()
        return None

    if not (1 <= port <= 65535):
        print(f"Warning: '{DENY_PORT_ANNOTATION}' out of range 1-
65535: {port}")
        sys.stdout.flush()
        return None

    return port

def remove_port_from_deployment(namespace, deployment_name,
denied_port):
    try:
        dep =
apps_v1.read_namespaced_deployment(name=deployment_name,
namespace=namespace)
```

```
    except ApiException as e:
        if e.status in (403, 404):
            print("Hint: verify RBAC and that the Deployment
exists.")
        print(f"Error: read_namespaced_deployment failed: {e}");
sys.stdout.flush()
        return False

    d = dep.to_dict()
    containers = (d.get("spec", {}).get("template", {}).get("spec",
{}).get("containers") or [])
    if not containers:
        print("DEBUG: Deployment has no containers.");
sys.stdout.flush()
        return False

    ops = []
    # build JSON Patch ops: wipe ports [] for any container that
exposes denied_port
    for idx, c in enumerate(containers):
        ports = c.get("ports") or []
        # find indices of entries whose container_port/containerPort
== denied_port
        rm_idxs = []
        for j, p in enumerate(ports):
            raw = p.get("container_port", p.get("containerPort"))
            try:
                cp = int(raw) if raw is not None else None
            except Exception:
                cp = None
            if cp == int(denied_port):
                rm_idxs.append(j)

        # JSON Patch: remove matching items from the list
        # reverse is necessary because if we remove index 0 first
from port array, then later indexes will shift down and not match our
rm_idxs array
        for j in sorted(rm_idxs, reverse=True):
            ops.append({
```

```
                "op": "remove",
                "path":
f"/spec/template/spec/containers/{idx}/ports/{j}"
            })

    if not ops:
        print(f"DEBUG: No container exposed port {denied_port}. No
patch built."); sys.stdout.flush()
        return False

    # ensure annotations map exists, then add/replace a timestamp to
force new RS
    ops.append({
        "op": "add",
        "path": "/spec/template/metadata/annotations",
        "value": d.get("spec", {}).get("template",
{}).get("metadata", {}).get("annotations", {}) or {}
    })
    # escape "/" in JSON Patch paths with "~1"
    ops.append({
        "op": "add",  # add acts like upsert for a missing key
        "path": "/spec/template/metadata/annotations/deny-auto~1ts",
        "value": str(int(time.time()))
    })

    # apply with JSON Patch content-type; retry lightly on conflicts
    for delay in (0, 0.2, 0.5):
        try:
            apps_v1.patch_namespaced_deployment(
                name=deployment_name,
                namespace=namespace,
                body=ops,

            )
            print(f"DEBUG: JSON patched deployment
'{deployment_name}' to clear ports for containers exposing
{denied_port}.")
            sys.stdout.flush()
            return True
```

```
        except ApiException as e:
            if e.status == 409:
                time.sleep(delay); continue
            print(f"Error: JSON patch failed: {e}");
sys.stdout.flush()
            return False

    return False



def run_automation():
    print(f"Starting Otterize Denial Automation (continuous watch)
for namespace: {NAMESPACE_TO_WATCH}")
    sys.stdout.flush()

    reconnect_attempts = 0
    MAX_RECONNECT_ATTEMPTS = 5  # Limit rapid reconnects if
persistent issue
    RECONNECT_DELAY_SECONDS = 5  # Delay between reconnect attempts

    while True:  # Keep trying to establish and maintain the watch
        w = watch.Watch()
        try:
            print(f"DEBUG: Attempting to establish watch on
ClientIntents in namespace {NAMESPACE_TO_WATCH} (Attempt
{reconnect_attempts + 1})...")
            sys.stdout.flush()
            # Stream events for ClientIntents
            for event in w.stream(
                custom_objects_api.list_namespaced_custom_object,
                group="k8s.otterize.com",
                version="v2beta1",
                namespace=NAMESPACE_TO_WATCH,
                plural="clientintents"
            ):
                event_type = event['type']
                obj = event['object']
                intent_name = obj['metadata']['name']
```

```
                intent_namespace = obj['metadata']['namespace']

                reconnect_attempts = 0  # Reset attempts on
successful event processing

                print(f"\nProcessing event '{event_type}' for
ClientIntent '{intent_name}' in namespace '{intent_namespace}'")
                sys.stdout.flush()

                if event_type == "ADDED" or event_type == "MODIFIED":
                    denied_port =
get_denied_port_from_annotation(obj)
                    if denied_port:
                        print(f"ClientIntent '{intent_name}' has
'{DENY_PORT_ANNOTATION}' annotation. Denied port: {denied_port}")
                        sys.stdout.flush()

                        workload_name = None
                        if 'spec' in obj and 'workload' in
obj['spec'] and 'name' in obj['spec']['workload']:
                            workload_name =
obj['spec']['workload']['name']

                        if not workload_name:
                            print(f"Warning: ClientIntent
'{intent_name}' does not specify a workload name. Skipping.")
                            sys.stdout.flush()
                            continue

                        try:
                            # Before attempting patch, ensure
deployment exists

apps_v1.read_namespaced_deployment(workload_name, intent_namespace)
                        except client.ApiException as e:
                            if e.status == 404:
                                print(f"Warning: Deployment
'{workload_name}' not found for ClientIntent '{intent_name}'.
Skipping patch.")
```

```
                                sys.stdout.flush()
                                continue
                            else:
                                print(f"Error reading deployment
'{workload_name}': {e}")
                                sys.stdout.flush()
                                continue

                        success =
remove_port_from_deployment(intent_namespace, workload_name,
denied_port)
                        if success:
                            print(f"Automation for ClientIntent
'{intent_name}' completed for {workload_name}.")
                            sys.stdout.flush()
                        else:
                            print(f"Automation for ClientIntent
'{intent_name}' failed to remove port {denied_port}.")
                            sys.stdout.flush()
                    else:
                        print(f"ClientIntent '{intent_name}' does not
have the '{DENY_PORT_ANNOTATION}' annotation or it's malformed.")
                        sys.stdout.flush()

                elif event_type == "DELETED":
                    print(f"ClientIntent '{intent_name}' was deleted.
No action taken for port re-addition based on current script logic.")
                    sys.stdout.flush()

        except client.ApiException as e:
            print(f"ERROR: Kubernetes API error during watch: {e}.
Attempting to reconnect in {RECONNECT_DELAY_SECONDS} seconds...")
            sys.stdout.flush()
            reconnect_attempts += 1
            if reconnect_attempts >= MAX_RECONNECT_ATTEMPTS:
                print(f"ERROR: Max reconnect attempts
({MAX_RECONNECT_ATTEMPTS}) reached. Exiting due to persistent API
errors.")
                sys.stdout.flush()
```

```
                sys.exit(1)  # Exit if persistent API errors
            time.sleep(RECONNECT_DELAY_SECONDS)  # Wait before
retrying

        except Exception as e:
            print(f"ERROR: An unexpected error occurred during watch:
{e}. Attempting to reconnect in {RECONNECT_DELAY_SECONDS}
seconds...")
            sys.stdout.flush()
            reconnect_attempts += 1
            if reconnect_attempts >= MAX_RECONNECT_ATTEMPTS:
                print(f"ERROR: Max reconnect attempts
({MAX_RECONNECT_ATTEMPTS}) reached. Exiting due to persistent
unexpected errors.")
                sys.stdout.flush()
                sys.exit(1)  # Exit if persistent unexpected errors
            time.sleep(RECONNECT_DELAY_SECONDS)  # Wait before
retrying

        finally:
            w.stop()
            print("DEBUG: Watch connection stopped, attempting to
restart.")
            sys.stdout.flush()


def main():
    run_automation()


if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(e); sys.exit(1)
```

A.2.5 Job/Deployment files:

Defines a Kubernetes Job that runs the deny-automation controller once to perform a single port-denial operation.

```
   apiVersion: batch/v1
kind: Job
metadata:
  name: deny-automation-once
  namespace: otterize-automation
spec:
  template:
    spec:
      serviceAccountName: deny-automation-controller
      restartPolicy: Never
      containers:
        - name: controller
          image: deny-auto:local
          env:
            - name: NAMESPACE_TO_WATCH
              value: "otterize-auto"
```

Deploys the custom deny-automation controller with required RBAC, environment variables, and runtime configuration.

```
   apiVersion: apps/v1
kind: Deployment
metadata:
  name: deny-automation-controller
  namespace: otterize-automation
  labels:
    app: deny-automation
spec:
  replicas: 1
  selector:
    matchLabels:
      app: deny-automation
  template:
    metadata:
```

```
      labels:
        app: deny-automation
    spec:
      serviceAccountName: deny-automation-controller # must match
your rbac.yaml
      containers:
        - name: controller
          image: deny-auto:local # replace if needed
          imagePullPolicy: IfNotPresent
          env:
            - name: NAMESPACE_TO_WATCH
              value: "otterize-auto" # adjust if different
```

A.3 Additional Scripts:

A.3.1 Resetting Environment to Baseline

Bash script for resetting kubernetes cluster to the initial state

```
   #!/usr/bin/env bash
set -euo pipefail

# Path to your original manifest
ALL_YAML="./all.yaml"
# Namespace used in your all.yaml
NS="otterize-auto"
NS_AUTOMATION="otterize-automation"
JOB_NAME="deny-automation-once"
DEPLOY_NAME="deny-automation-controller"


echo "[1/5] Remove any ClientIntents in test namespace (clears deny
rules)"
kubectl delete clientintents.k8s.otterize.com -n "$NS" --all --
```

```
ignore-not-found

echo "[2/5] Remove any Jobs from the namespace"
kubectl delete job "$JOB_NAME" -n "$NS_AUTOMATION" --ignore-not-found

echo "[3/5] Delete existing Deployment"
kubectl delete deployment "$DEPLOY_NAME" -n "$NS_AUTOMATION" --
ignore-not-found

echo "[4/5] Re-apply original resources"
kubectl apply -f "$ALL_YAML"

echo "[5/5] Restart server Deployment to pick up baseline spec"
kubectl rollout restart deployment/server -n "$NS"
kubectl rollout status deployment/server -n "$NS" --timeout=120s


echo "Baseline restored: server listening on port 80 and reachable
from client."
```